



第11章 搜索策略



《算法导论》第16章

计算机围棋

策略网络：给定当前局面，预测并采样下一步的走棋。

快速走子：目标和策略网络一样，但在适当牺牲走棋质量的条件下速度要比策略网络快很多

价值网络：给定当前局面，估计是白胜概率大还是黑胜概率大。

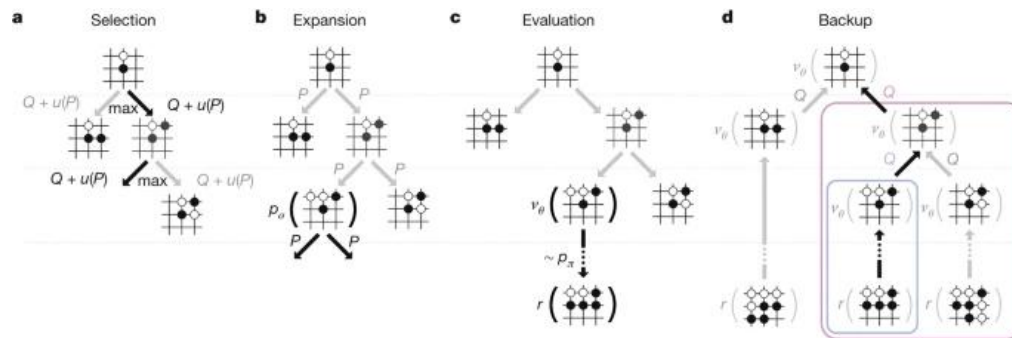
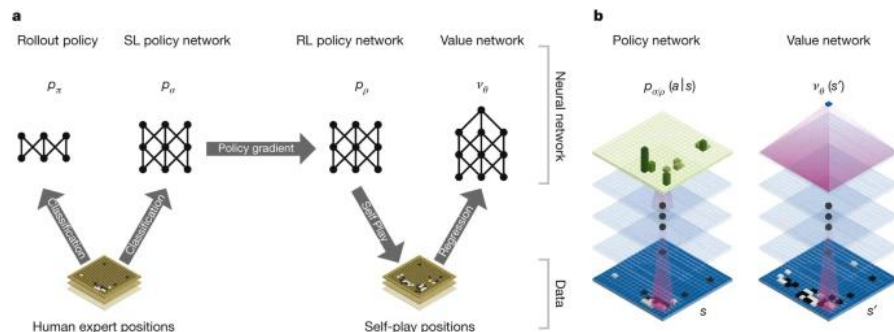
蒙特卡罗搜索：把这三个部分连起形成一个完整的系统。



KO



李世石



神经网络训练准则及其体系结构

蒙特卡罗树搜索

Silver D, Huang A, Maddison C J, et al. Mastering the game of Go with deep neural networks and tree search[J]. Nature, 2016, 529(7587): 484-489.

本讲内容

11.1 暴力美学：搜索漫谈

11.2 深度优先与广度优先

11.3 搜索的优化

11.4 剪枝方法论与人员安排问题

11.5 旅行商问题

11.6 A*算法

布尔表达式可满足性问题

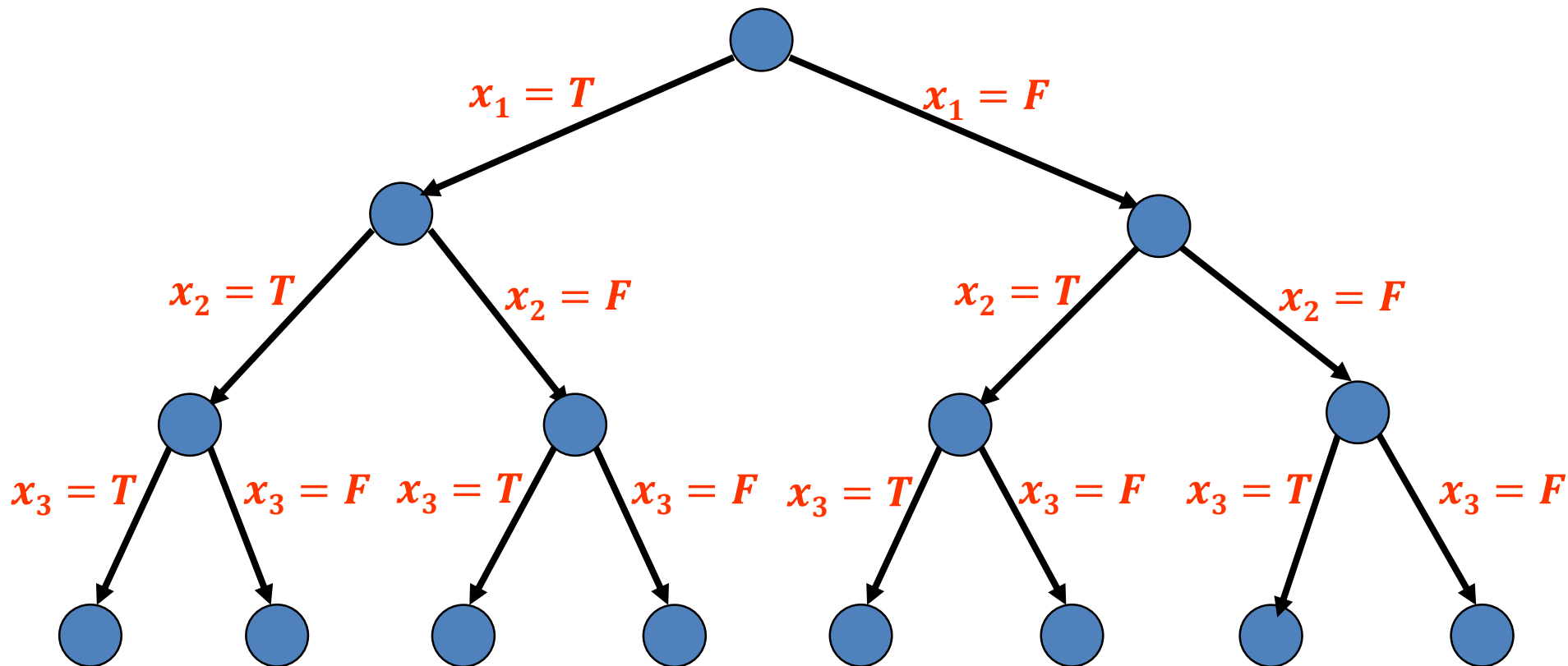
- 问题的定义

- 输入: n 个布尔变量 $x_1, x_2, \dots, x_n$, 关于 $x_1, x_2, \dots, x_n$ 的布尔表达式
- 输出: 是否存在一个 $x_1, x_2, \dots, x_n$ 的一种赋值, 使得布尔表达式为真

布尔表达式是布尔运算量和逻辑运算符按一定语法规则组成的式子, 它最终只有true (真) 和false (假) 两个取值。

问题表示为树搜索问题

- 通过不断地为赋值集合分类来建立树
(以三个变量 x_1, x_2, x_3 为例)



8-Puzzle问题

- 问题的定义

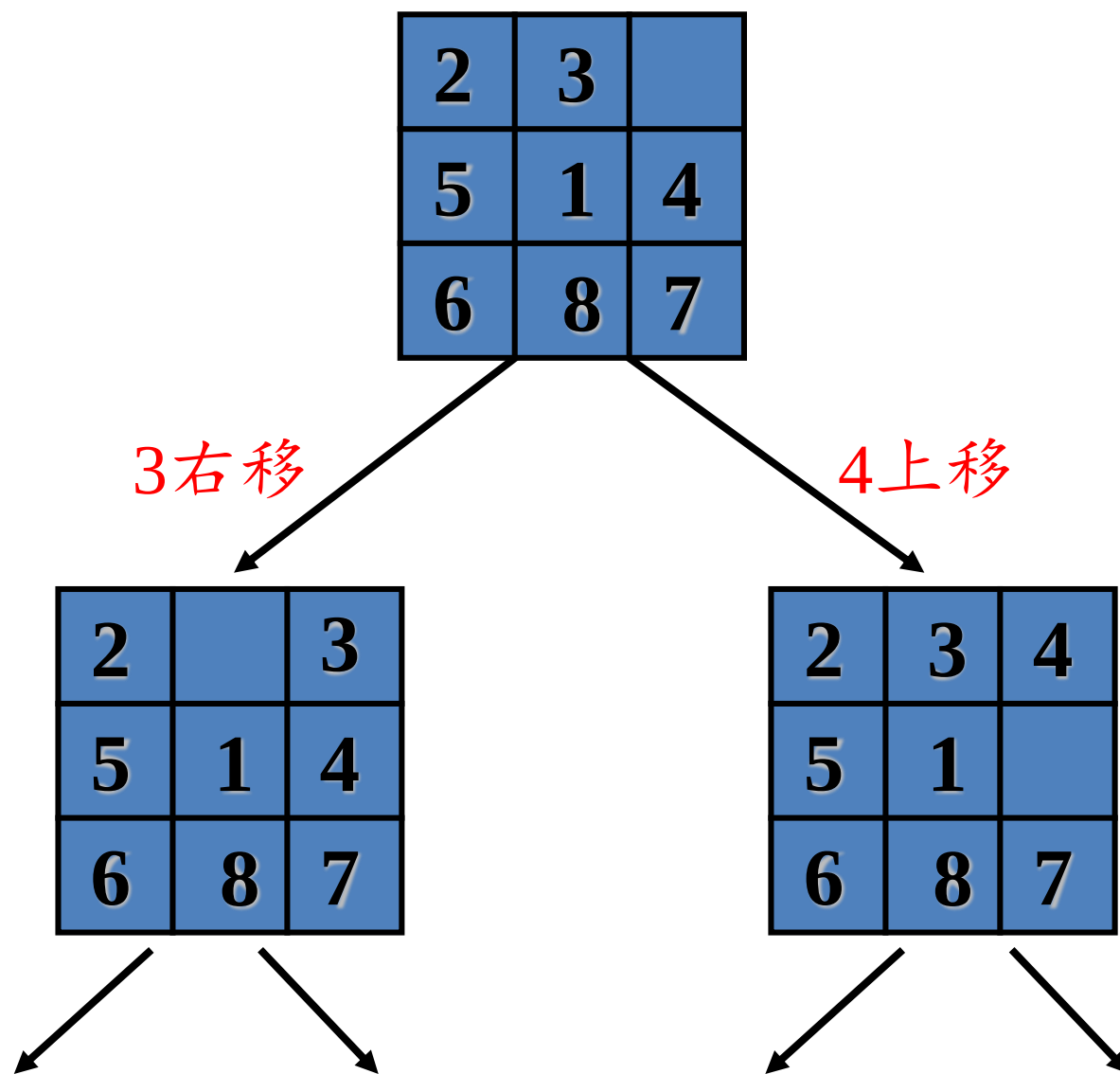
- 输入：具有8个编号小方块的魔方

2	3	
5	1	4
6	8	7

- 输出：移动系列，经过这些移动，魔方达如下状态

1	2	3
8		4
7	6	5

问题表示为树搜索问题



Hamiltonian环问题

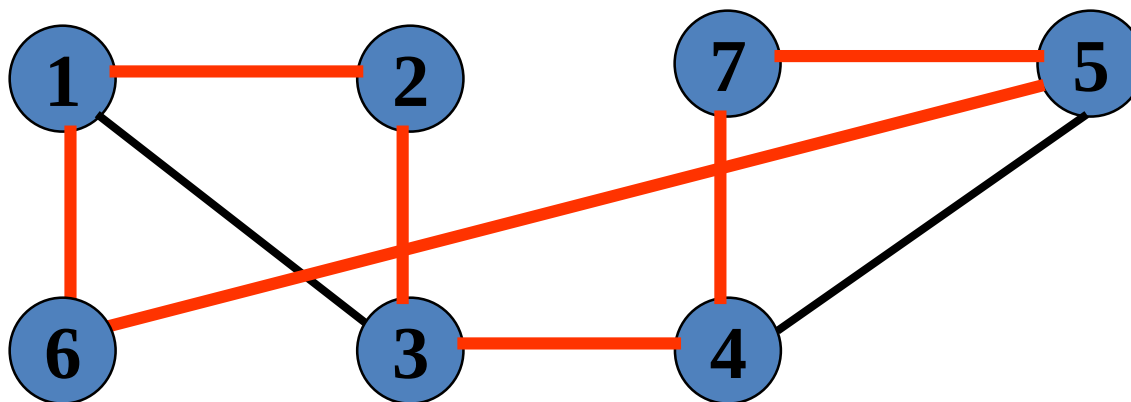
- 问题定义

- 输入: 具有 n 个结点的连通图 $G = (V, E)$
- 输出: G 中是否具有Hamiltonian环

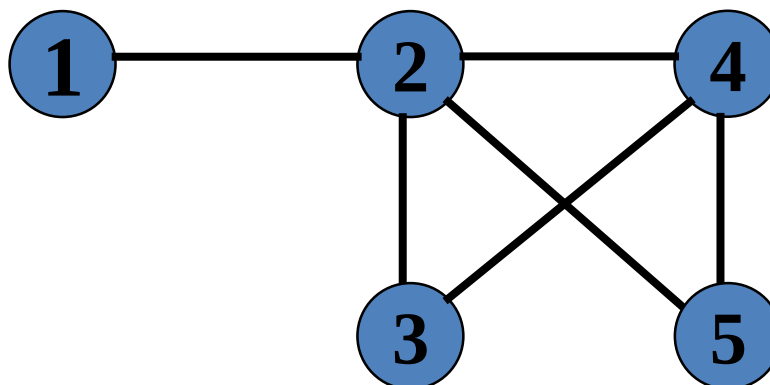
沿着 G 的 n 条边经过每个结点通过且仅通过一次，并回到起始结点的环称为 G 的一个Hamiltonian环。

Hamiltonian环问题

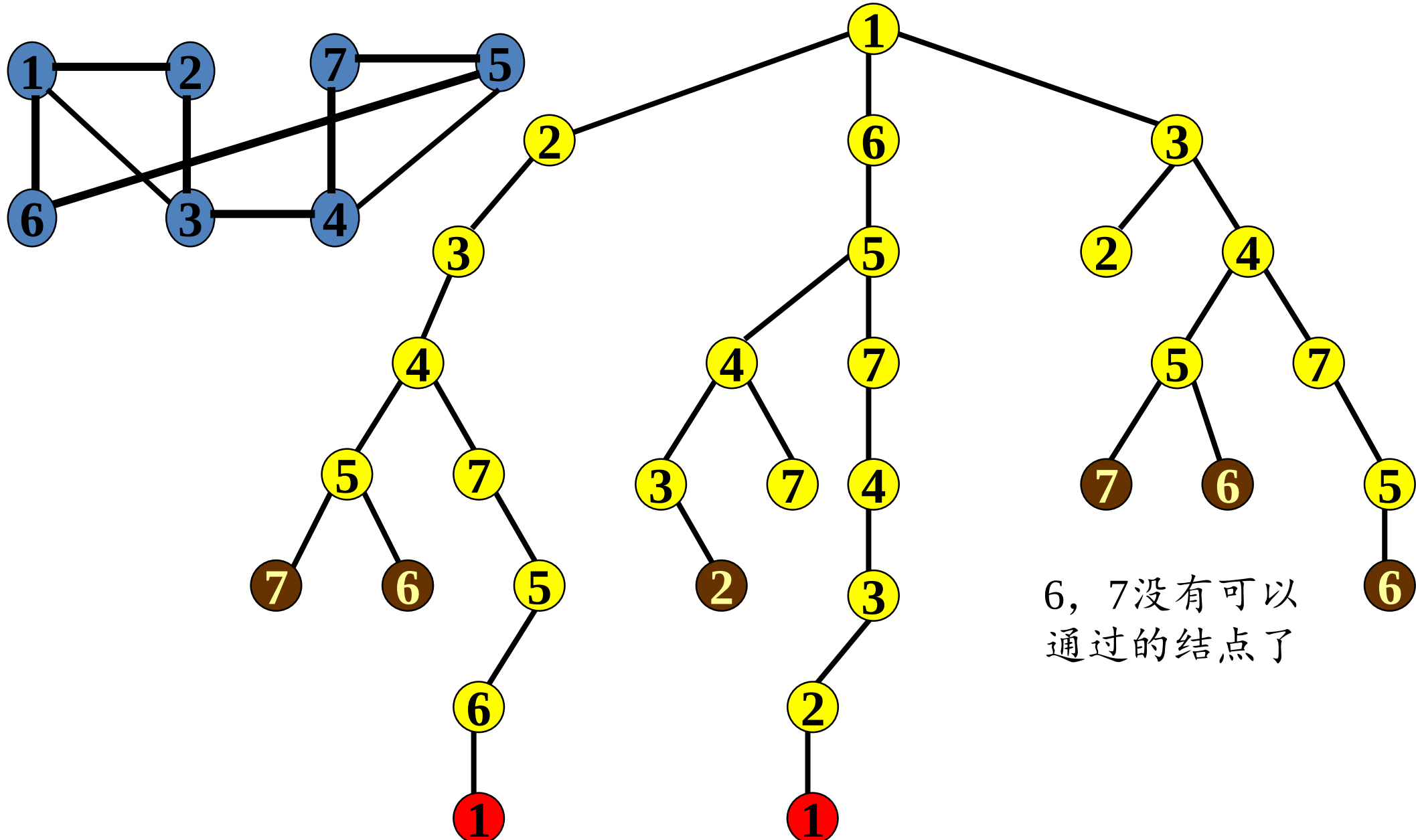
- 有Hamiltonian环图:



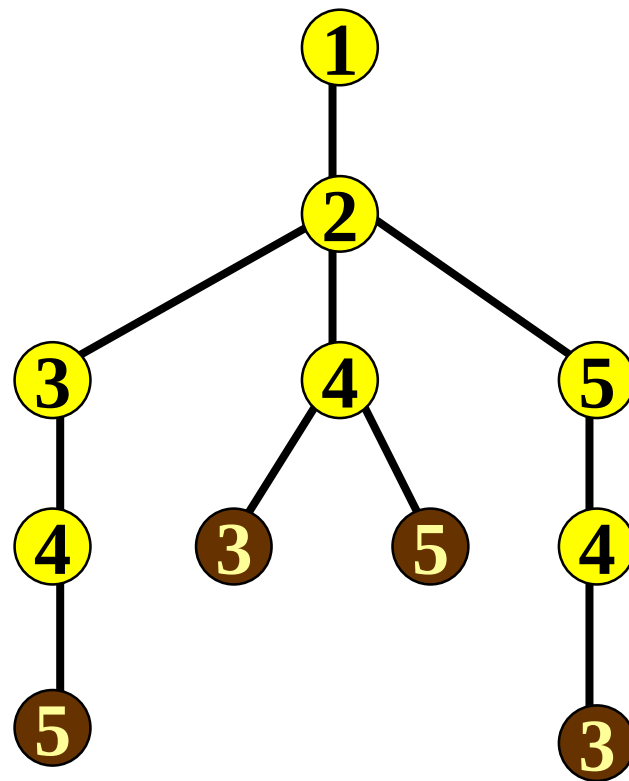
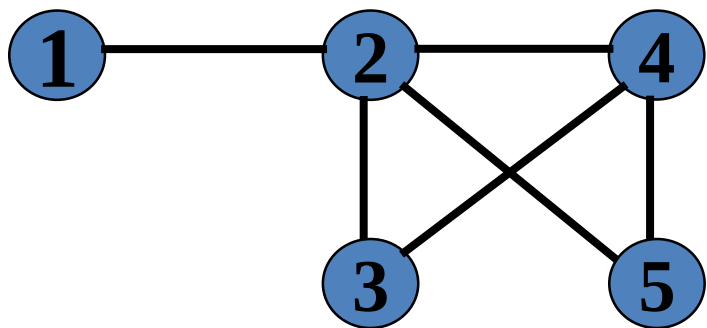
- 无Hamiltonian环图:



问题表示为树搜索问题



问题表示为树搜索问题



本讲内容

11.1 暴力美学：搜索漫谈

11.2 深度优先与广度优先

11.3 搜索的优化

11.4 剪枝方法论与人员安排问题

11.5 旅行商问题

11.6 A*算法

广度优先搜索

广度优先搜索算法框架：

1. 构造由根组成的队列 Q ;
2. **If** Q 的第一个元素 x 是目标结点 **Then** 停止;
3. **Else**
4. 从 Q 中删除 x , 把 x 的所有子结点加入 Q 的末尾;
5. **If** Q 空 **Then** 失败;
11. **Else** goto 2.

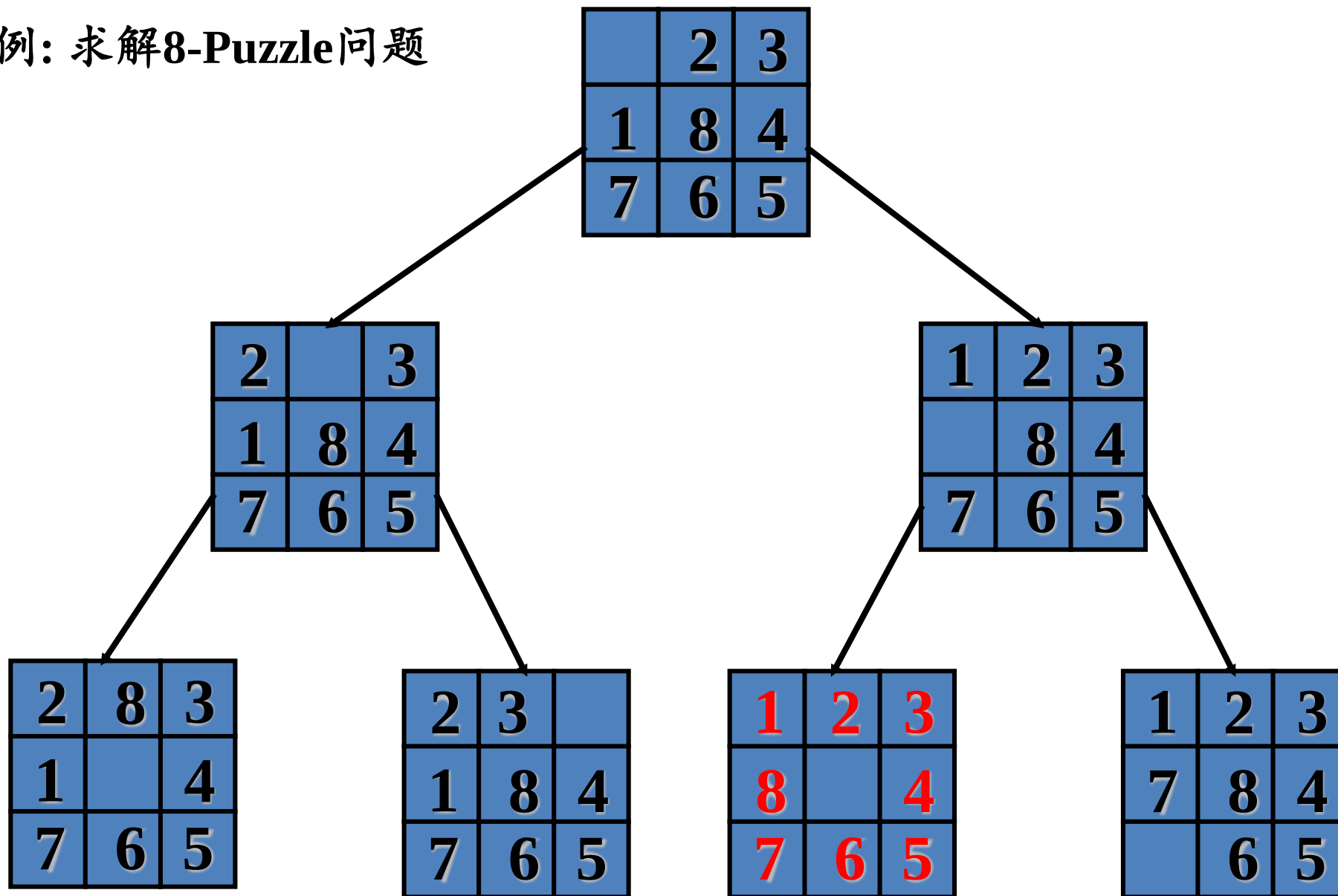
- 广度优先搜索的思想

广度优先搜索BFS遍历类似于树的按层次遍历。

- (1) 首先访问图中某一个指定的出发点 v_i ;
- (2) 然后依次访问 v_i 的所有邻接点 $v_{i1}, v_{i2} \dots v_{it}$;
- (3) 再依次以 $v_{i1}, v_{i2} \dots v_{it}$ 为顶点, 访问各顶点未被访问的邻接点, 依此类推, 直到图中所有顶点均被访问为止。
- (4) 若此时图中尚有顶点未被访问, 则另选图中一个未曾被访问的顶点作起始点, 重复上述过程, 直至图中所有顶点都被访问到为止。

广度优先搜索

- 例：求解8-Puzzle问题



深度优先搜索

深度优先搜索算法框架：

1. 构造一个由根构成的素栈 S ;
2. **If** Top(S)是目标结点 **Then** 停止;
3. Pop(S) /*取出栈顶元素*/
4. 把Top(S)的所有子结点压入栈顶;
5. **If** S 空 **Then** 失败;
11. **Else** goto 2.

• 算法的基本思想

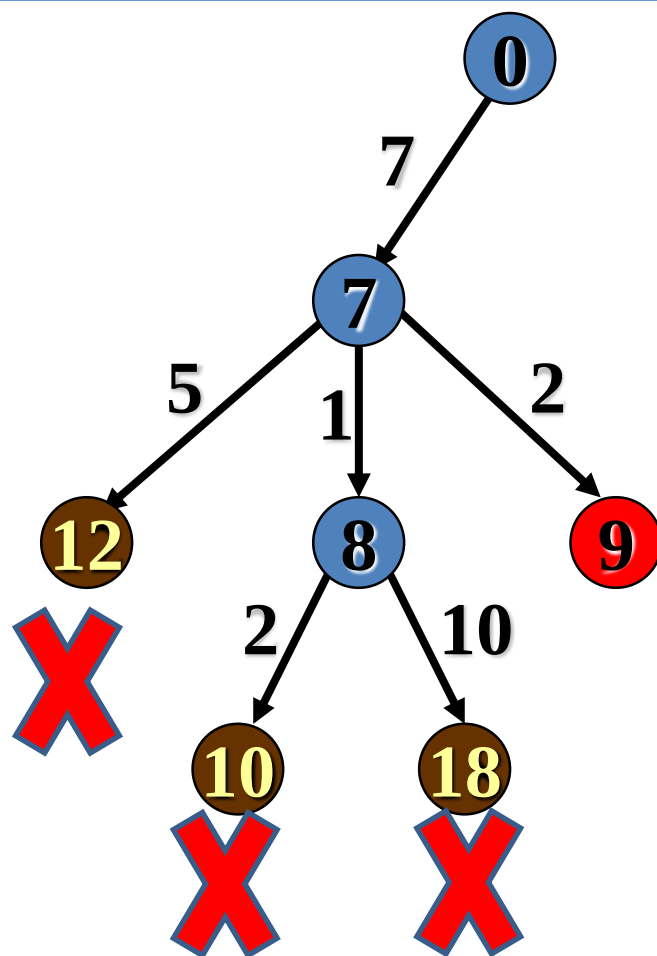
- (1)首先访问图中某一个顶点 V_i ，以该顶点为出发点;
- (2)任选一个与顶点 V_i 邻接的未被访问的顶点 V_j ；访问 V_j ;
- (3)以 V_j 为新的出发点继续进行深度优先搜索，直至图中所有和 V_i 有路径的顶点均被访问到。

深度优先搜索

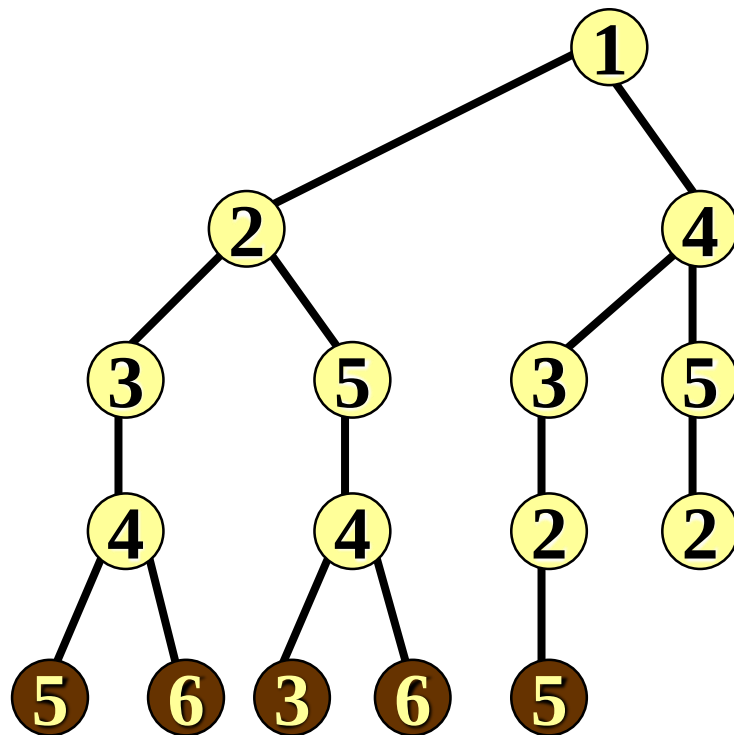
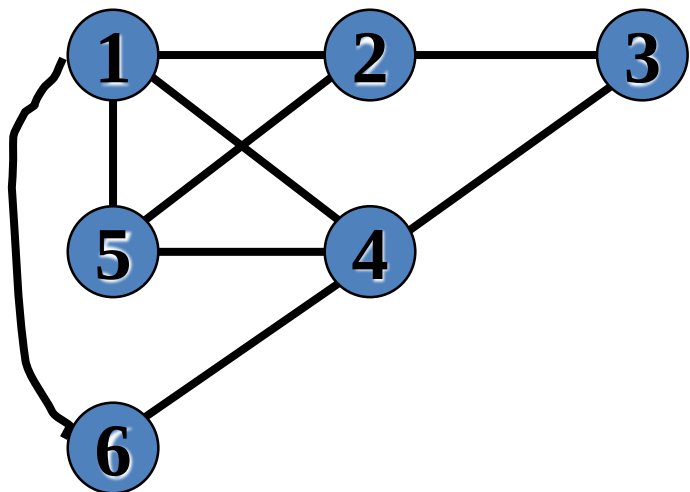
- 例：求解子集和问题

输入: $S = \{7, 5, 1, 2, 10\}$

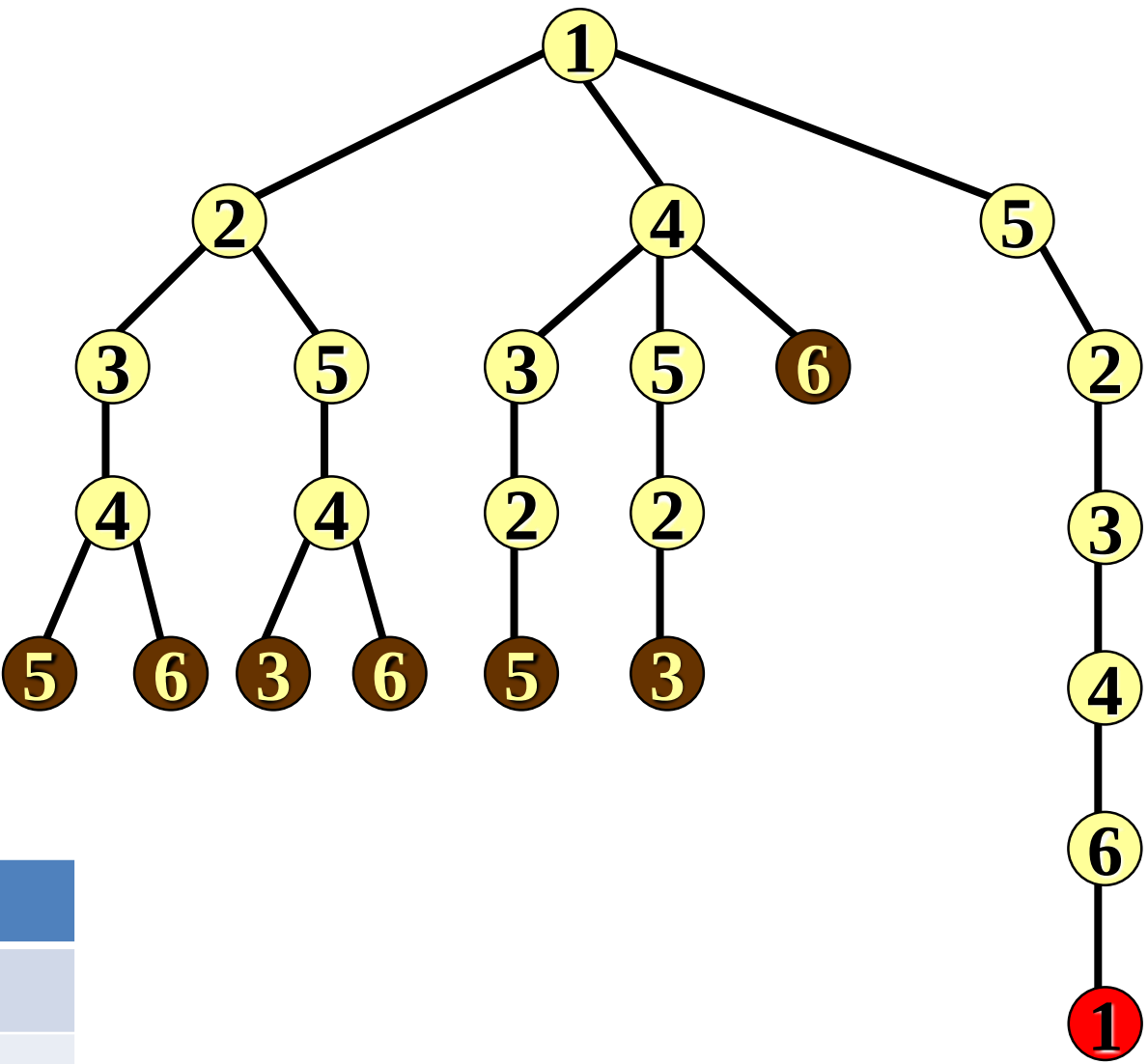
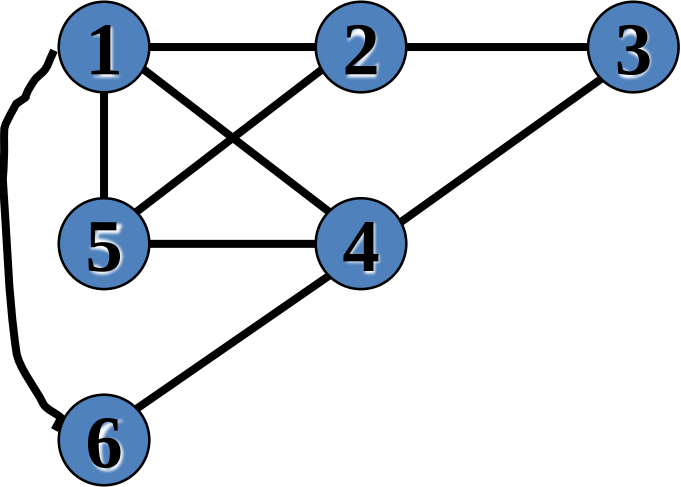
输出: 是否存在 $S' \subseteq S$, 使得 $Sum(S') = 9$



- 例：求解Hamiltonian环问题

[illegible]

• 例： 求解Hamiltonian环问题



3							
6	6		2	3	4	6	1
5	5	5	4	4	4	4	4
6	6	6	6	6	6	6	6

本讲内容

11.1 暴力美学：搜索漫谈

11.2 深度优先与广度优先

11.3 搜索的优化

11.4 剪枝方法论与人员安排问题

11.5 旅行商问题

11.6 A*算法

爬山法

- 基本思想

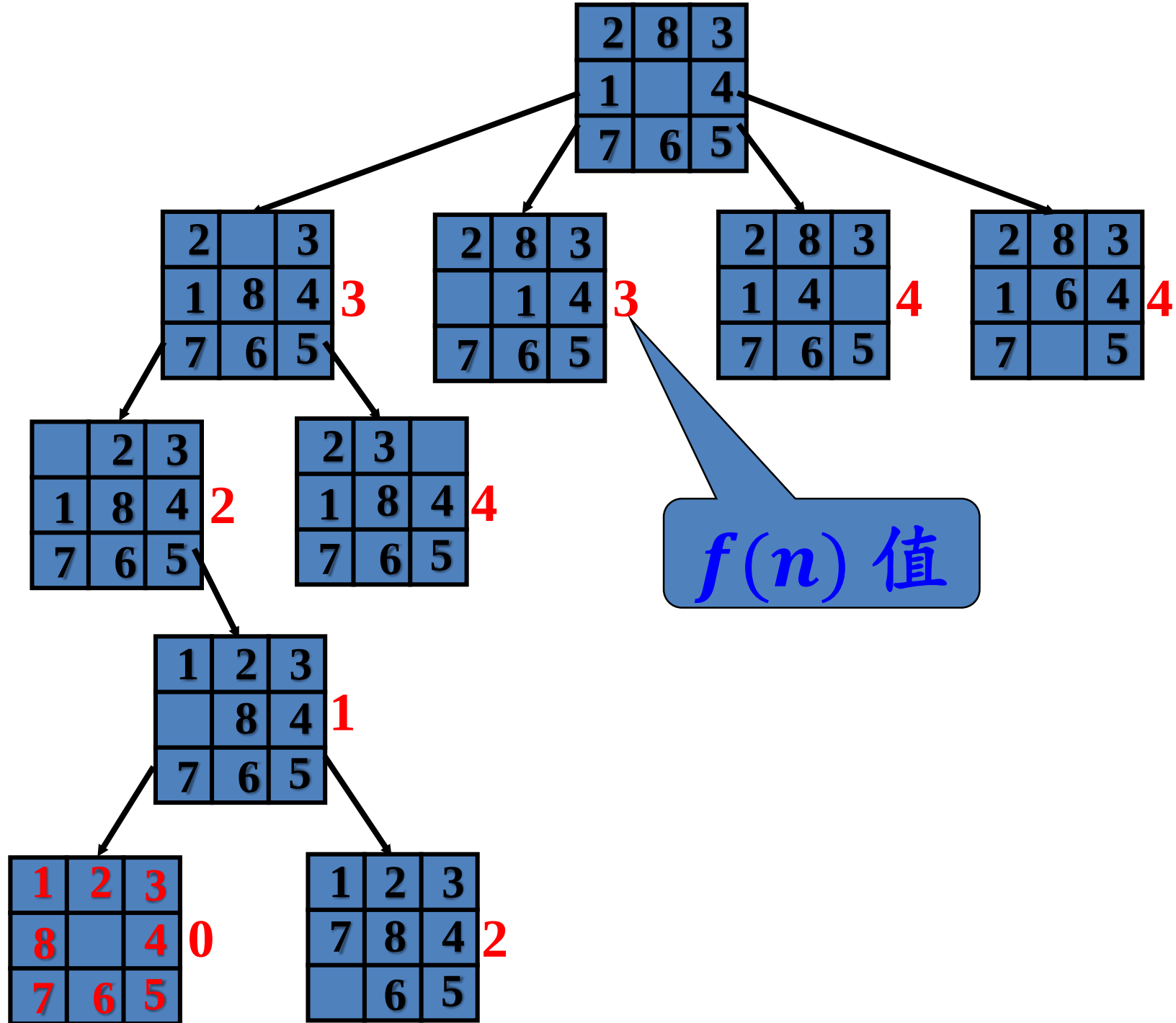
- 在深度优先搜索过程中，我们经常遇到多个结点可以扩展的情况，首先扩展哪个？
- 爬山策略使用贪心方法确定搜索的方向，是优化的深度优先搜索策略。
- 爬山策略使用启发式测度来排序结点扩展的顺序。
 - 使用什么启发式测度与问题本身相关。

深度优先搜索 + 启发式测度

爬山法

- 用8-Puzzle问题来说明爬山策略的思想
 - 启发式测度函数: $f(n) = W(n)$, $W(n)$ 是结点 n 中处于错误位置的方块数。
 - 例如, 如果结点 n 如下, 则 $f(n) = 3$, 因为方块1、2、8处于错误位置。

2	8	3
1		4
7	6	5



爬山法

爬山算法框架：

1. 构造一个由根构成的素栈 S ;
2. **If** Top(S)是目标结点 **Then** 停止;
3. Pop(S) /*取出栈顶元素*/
4. 把 S 的子结点按照其启发测度由大到小（由小到大）的顺序压入 S ;
5. **If** S 空 **Then** 失败;
11. **Else** goto 2.

- 爬山法并不能保证所得到的解是全局最优的。为什么？
- 虽不是全局最优的，但是一个可行解。
- 如何进一步改进？

Best-First 搜索

• 基本思想

- 结合深度优先和广度优先的优点。
- 根据一个评价函数，在目前产生的所有结点中选择具有最小（最大）评价函数值的结点进行扩展。
- 具有“全局优化”观念，而爬山策略仅具有局部优化观念。

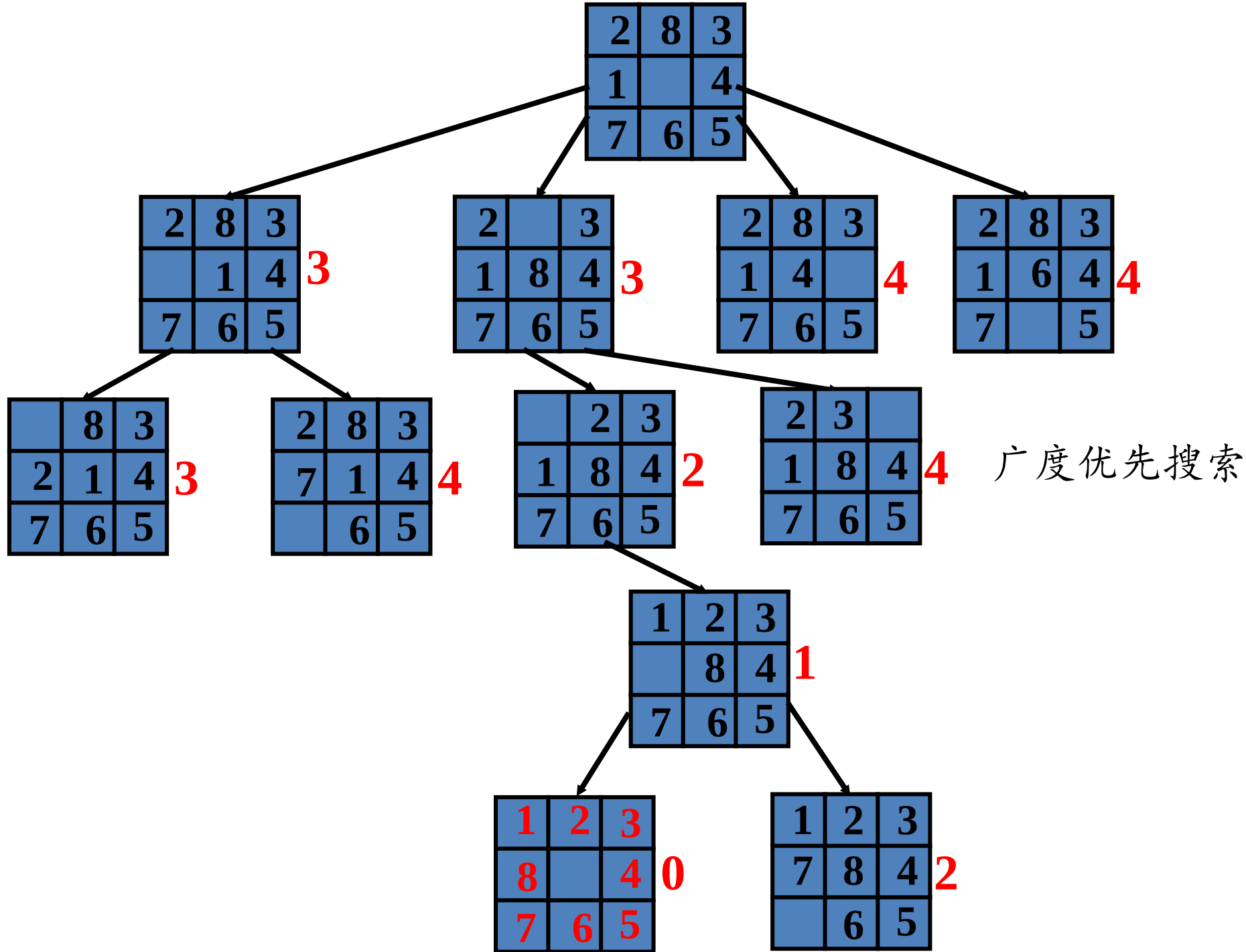
深度优先搜索 + 广度优先搜索 + 启发式测度

Best-First 搜索

Best-First算法框架：

1. 使用评价函数构造一个堆 H ，首先构造由根 r 组成的单元素堆；
2. **If** 堆 H 的根 r 是目标结点 **Then** 停止；
3. 从堆 H 中删除 r ，把 r 的子结点插入堆 H ；
4. **If** 堆 H 空 **Then** 失败；
5. **Else** goto 2.

堆 详见《算法导论》第6章



分支界限

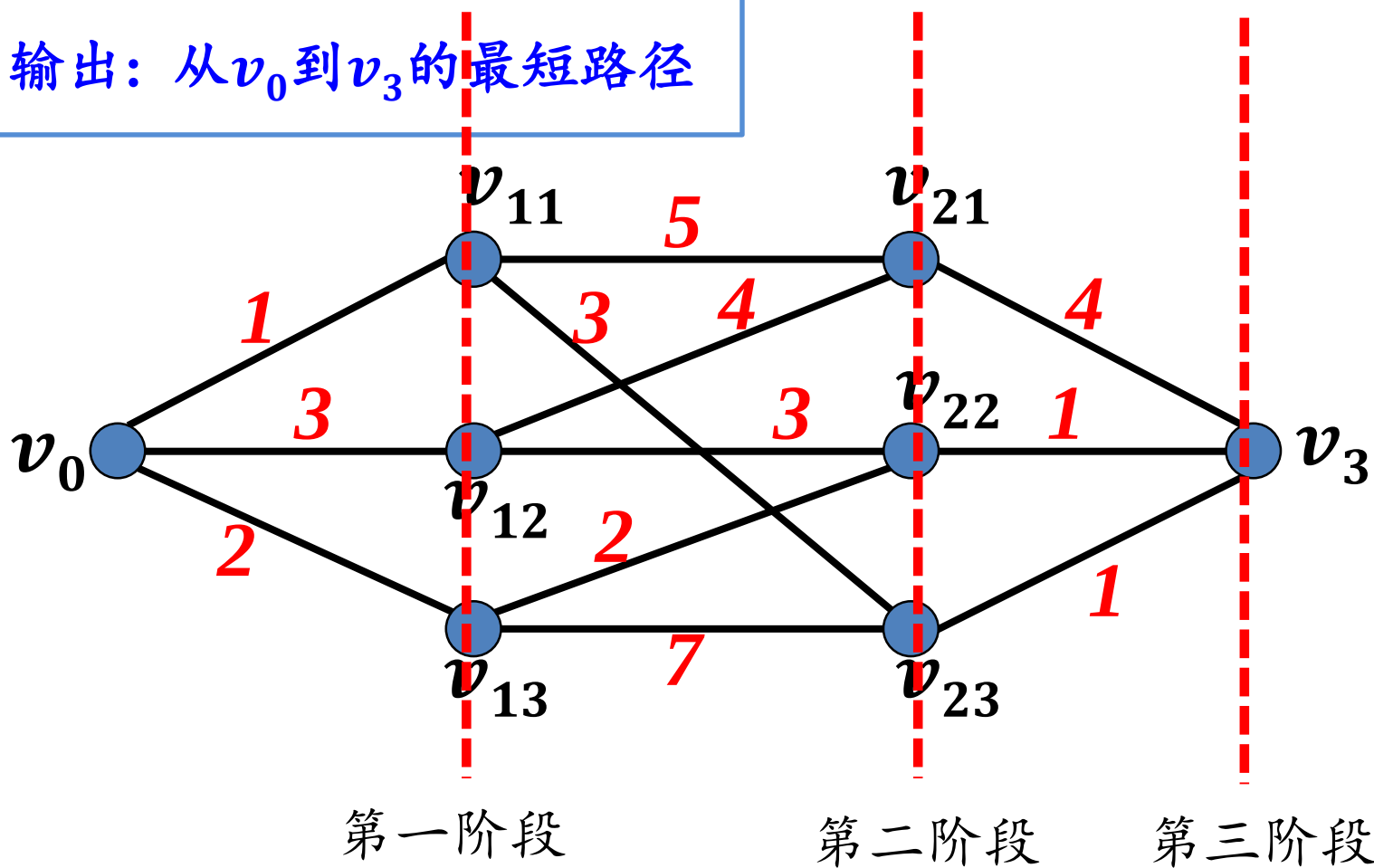
- 基本思想
 - 上述方法很难用于求解优化问题
 - 分支界限策略可以有效地求解组合优化问题
 - 思想：发现优化解的一个界限
 - 缩小解空间，提高求解的效率
- 举例说明分支界限策略的原理

分支界限

- 多阶段图搜索问题

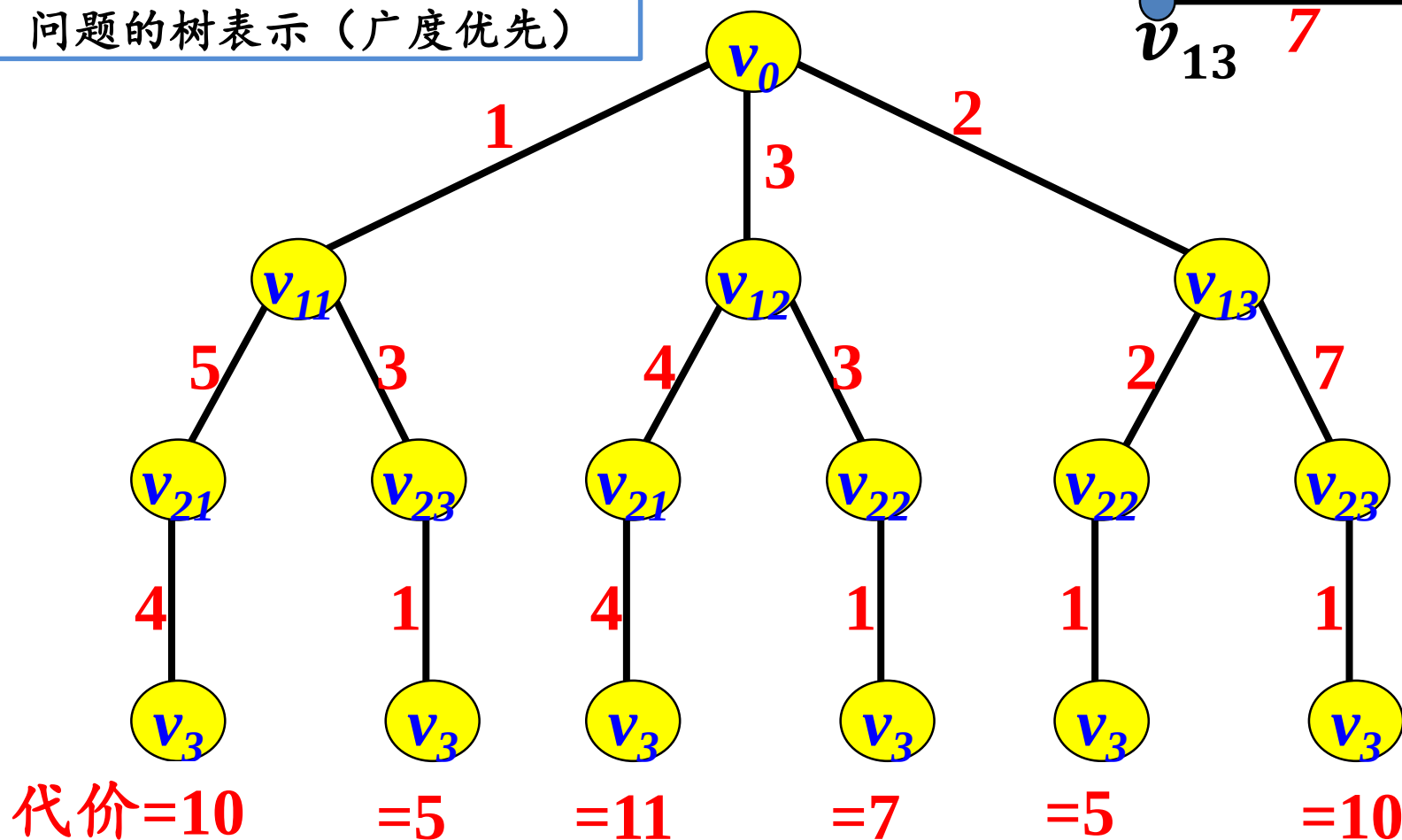
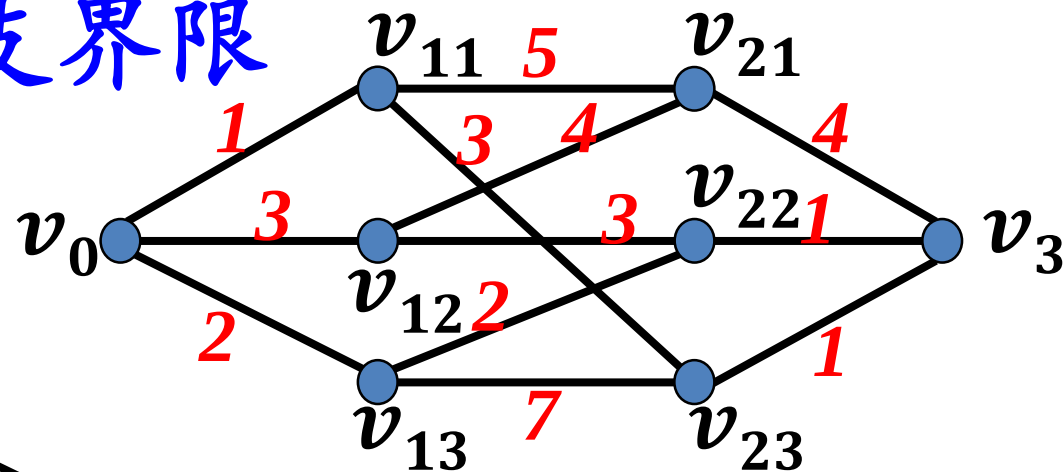
- 输入：多阶段图

- 输出：从 v_0 到 v_3 的最短路径



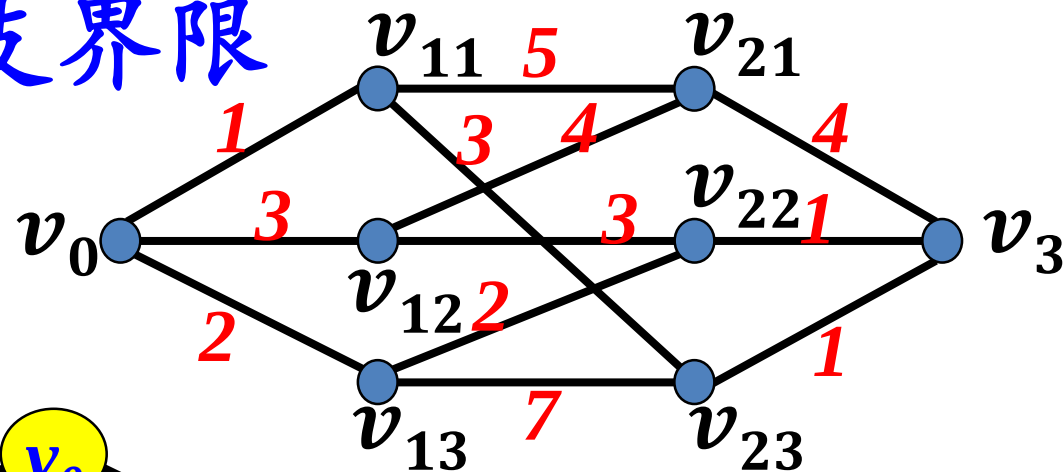
分支界限

- 多阶段图搜索问题
 - 输入: 多阶段图
 - 输出: 从 v_0 到 v_3 的最短路径
- 问题的树表示 (广度优先)

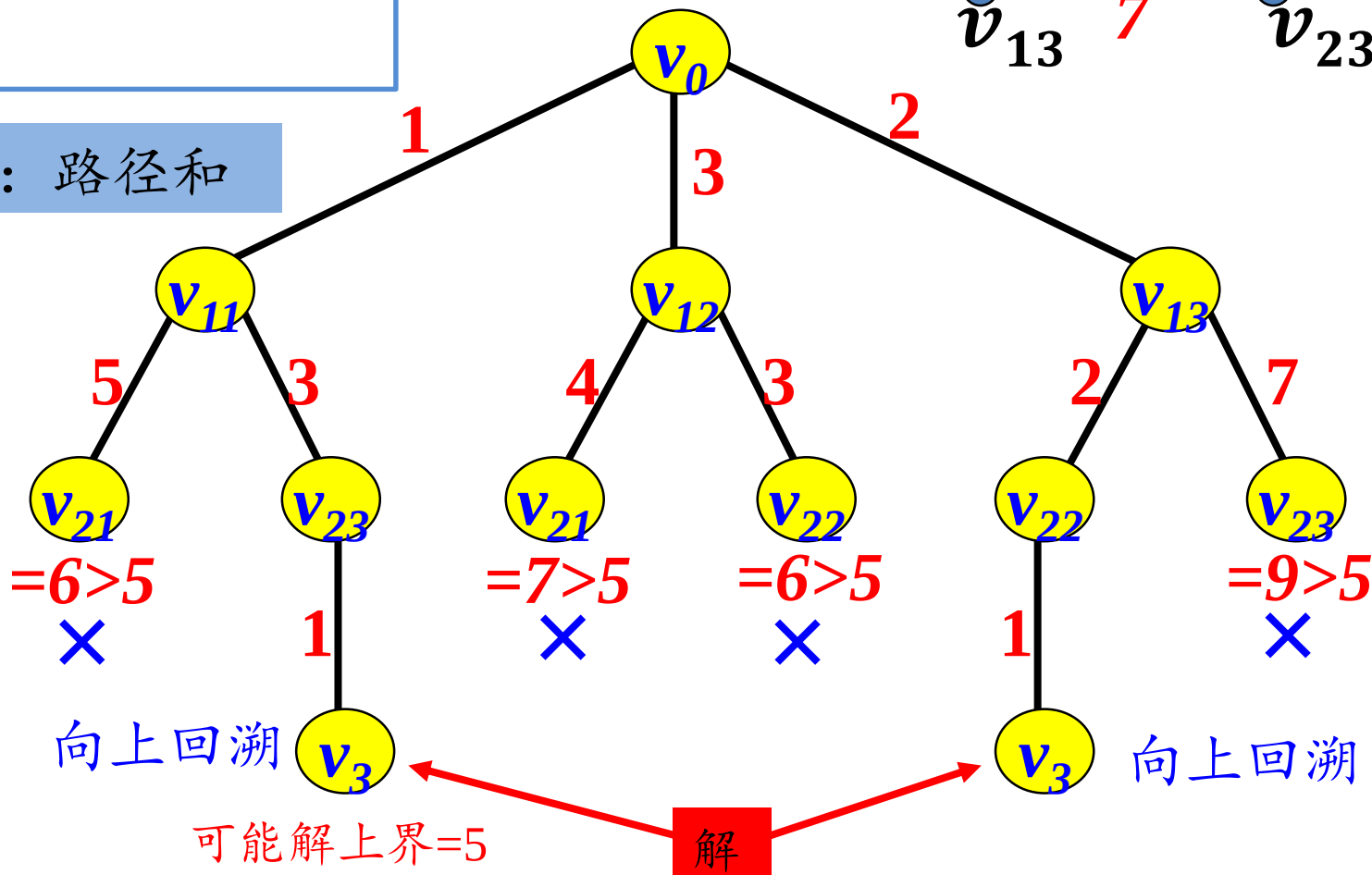


- 多阶段图搜索问题
 - 输入：多阶段图
 - 输出：从 v_0 到 v_3 的最短路径
- 问题的树表示
- 使用爬山策略进行分支界限搜索

分支界限



评价函数：路径和



分支界限

- 分支界限策略的原理
 - 产生分支的机制（使用前面的任意一种策略）
 - 产生一个界限（可以通过发现可能解）
 - 进行分支界限搜索，即剪除不可能产生优化解的分支

总结

- 爬山法：深度优先搜索 + 启发式测度
- **Best-First搜索**：深度优先搜索 + 广度优先搜索 + 启发式测度
- 分支界限：界限 + 剪枝

随堂测验

- 1、深度优先搜索使用栈作为数据结构 (✓)
- 2、使用贪心法能够求的最优解的问题一定能够使用搜索求得最优解 (✓)
- 3、分支界限法可以和爬山法结合在一起使用得到问题的最优解 (✓)
- 4、深度优先搜索的时间复杂性高于广度优先搜索 (✗)
- 5、Best-first搜索的效率一定比爬山法高 (✗)

本讲内容

11.1 暴力美学：搜索漫谈

11.2 深度优先与广度优先

11.3 搜索的优化

11.4 剪枝方法论与人员安排问题

11.5 旅行商问题

11.6 A*算法

人员安排问题的定义

- 输入

- 人的集合 $P = \{P_1, P_2, \dots, P_n\}$, $P_1 < P_2 < \dots < P_n$, 工作的集合 $J = \{J_1, J_2, \dots, J_n\}$, J 是偏序集合
- 矩阵 $[C_{ij}]$, C_{ij} 是工作 J_j 分配到 P_i 的代价

- 输出

- 矩阵 $[X_{ij}]$, $X_{ij} = 1$ 表示 P_i 被分配 J_j , $\sum_{ij} C_{ij} X_{ij}$ 最小
- 每个人被分配一种工作, 不同人分配不同工作

例. 给定 $P = \{P_1, P_2, P_3\}$, $J = \{J_1, J_2, J_3\}$, $J_1 \leq J_3$, $J_2 \leq J_3$.

$P_1 \rightarrow J_1$ 、 $P_2 \rightarrow J_2$ 、 $P_3 \rightarrow J_3$ 是可行解。

$P_1 \rightarrow J_1$ 、 $P_2 \rightarrow J_3$ 、 $P_3 \rightarrow J_2$ 不可能是解。

求解问题的思想

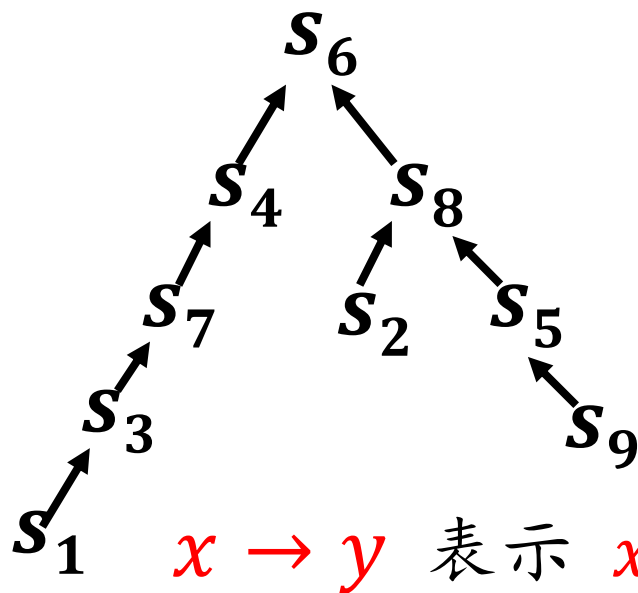
- 人员安排问题转化为树搜索问题
 - $J = \{J_1, J_2, \dots, J_n\}$ 的每个拓扑序列对应一个解
 - 用一个拓扑序列树把所有拓扑序列安排在一个树中，每个路径对应一个拓扑序列
- 问题转化：构造拓扑序列树
- 求解问题：使用分支界限法搜索拓扑序列树求解问题

拓扑序列是顶点活动网中将活动按发生的先后次序进行的一种排列。

转换为树搜索问题

- 拓朴排序

- 输入: 偏序集合($S, \leq$)
- 输出: S 的拓朴序列是 $\langle s_1, s_2, \dots, s_n \rangle$, 满足条件: 如果 $s_i \leq s_j$, 则 s_i 排在 s_j 的前面。
- 例



拓朴排序: $s_1 s_3 s_7 s_4 s_9 s_5 s_2 s_8 s_6$

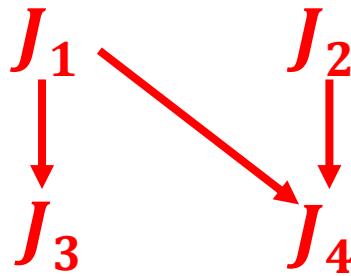
$x \rightarrow y$ 表示 $x \leq y$

转换为树搜索问题

- 问题的解空间

- 命题1.** $P_1 \rightarrow J_{k1}$ 、 $P_2 \rightarrow J_{k2}$ 、 $\dots$ 、 $P_n \rightarrow J_{kn}$ 是一个可能解，当且仅当 J_{k1} 、 J_{k2} 、 $\dots$ 、 J_{kn} 必是一个拓朴排序的序列。

例. $P = \{P_1, P_2, P_3, P_4\}$, $J = \{J_1, J_2, J_3, J_4\}$, J 的偏序如下:

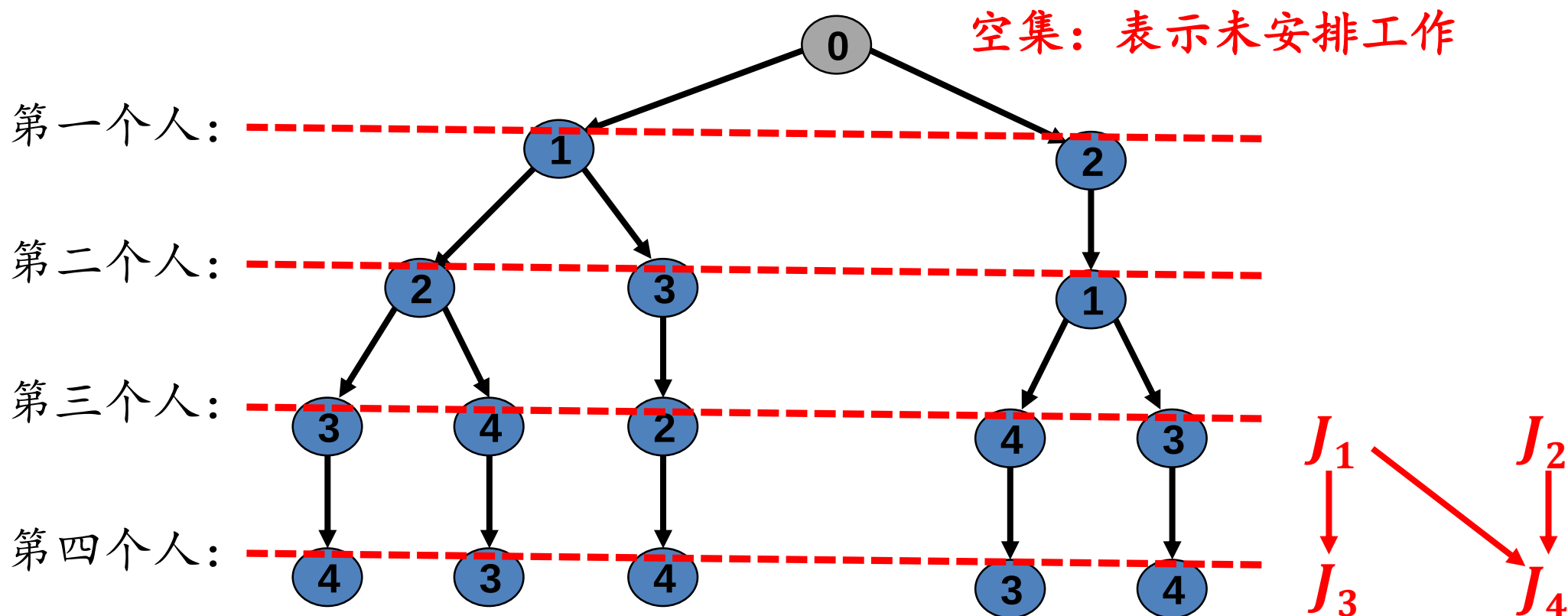


- 则 (J_1, J_2, J_3, J_4) 、 (J_1, J_2, J_4, J_3) 、 (J_1, J_3, J_2, J_4) 、 (J_2, J_1, J_3, J_4) 、 (J_2, J_1, J_4, J_3) 是拓朴排序列
- (J_1, J_2, J_4, J_3) 对应于 $P_1 \rightarrow J_1$ 、 $P_2 \rightarrow J_2$ 、 $P_3 \rightarrow J_4$ 、 $P_4 \rightarrow J_3$

问题的解空间是所有拓朴排序的序列集合，每个序列对应一个可能的解

转换为树搜索问题

- 问题的树表示（即用树表示所有拓扑排序序列）



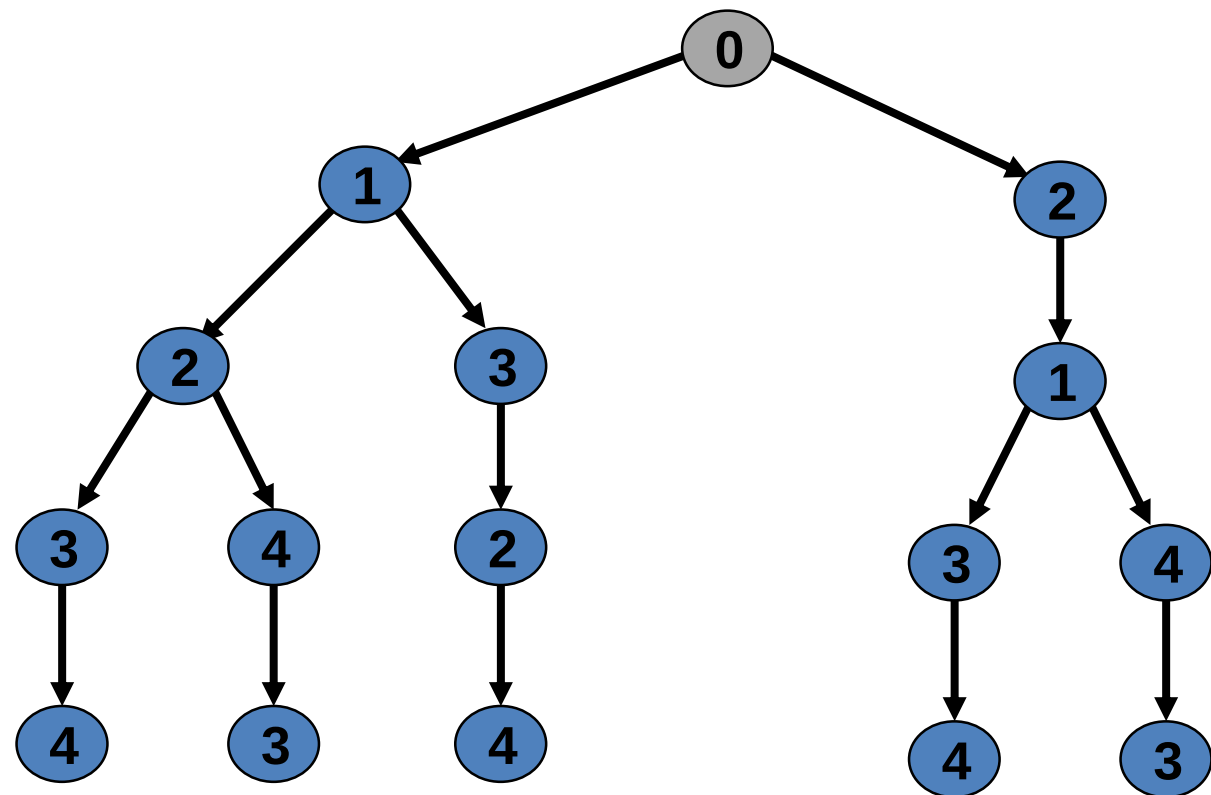
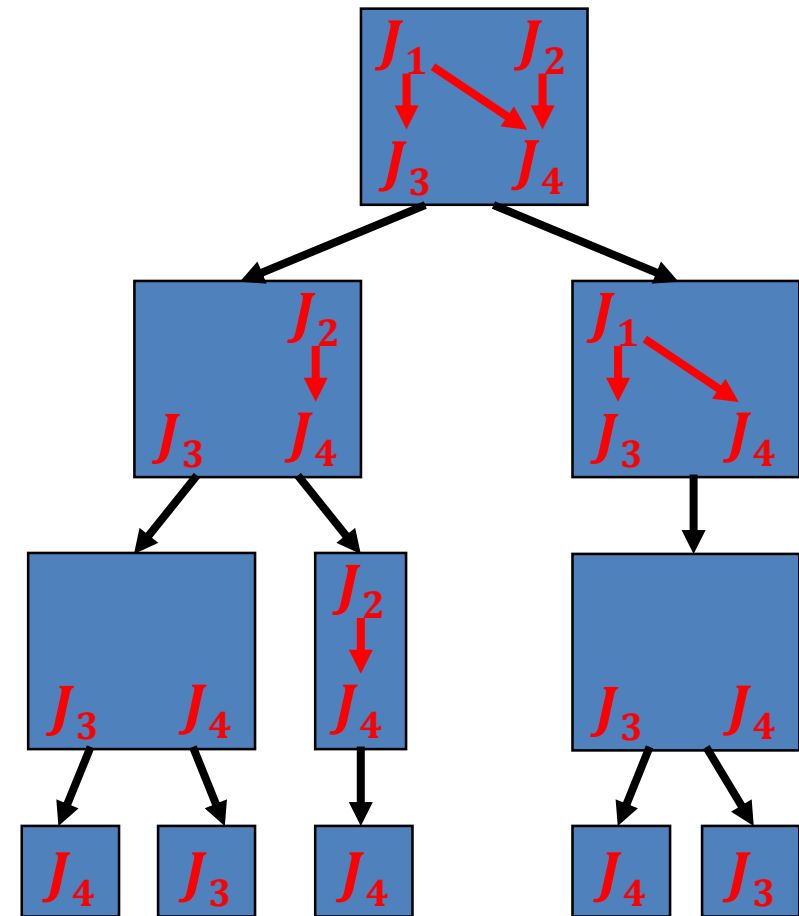
转换为树搜索问题

拓朴序列树的生成算法：

输入： 偏序集合 S ，树根root。

输出： 由 S 的所有拓朴排序序列构成的树。

1. 生成树根root；
2. 选择偏序集中**没有前序元素**的所有元素，作为root的子结点；
3. **For** root的每个子结点 v **Do**
4. $S = S - \{v\}$;
5. 把 v 作为根，**递归地**处理 S 。



拓朴序列树的生成算法:

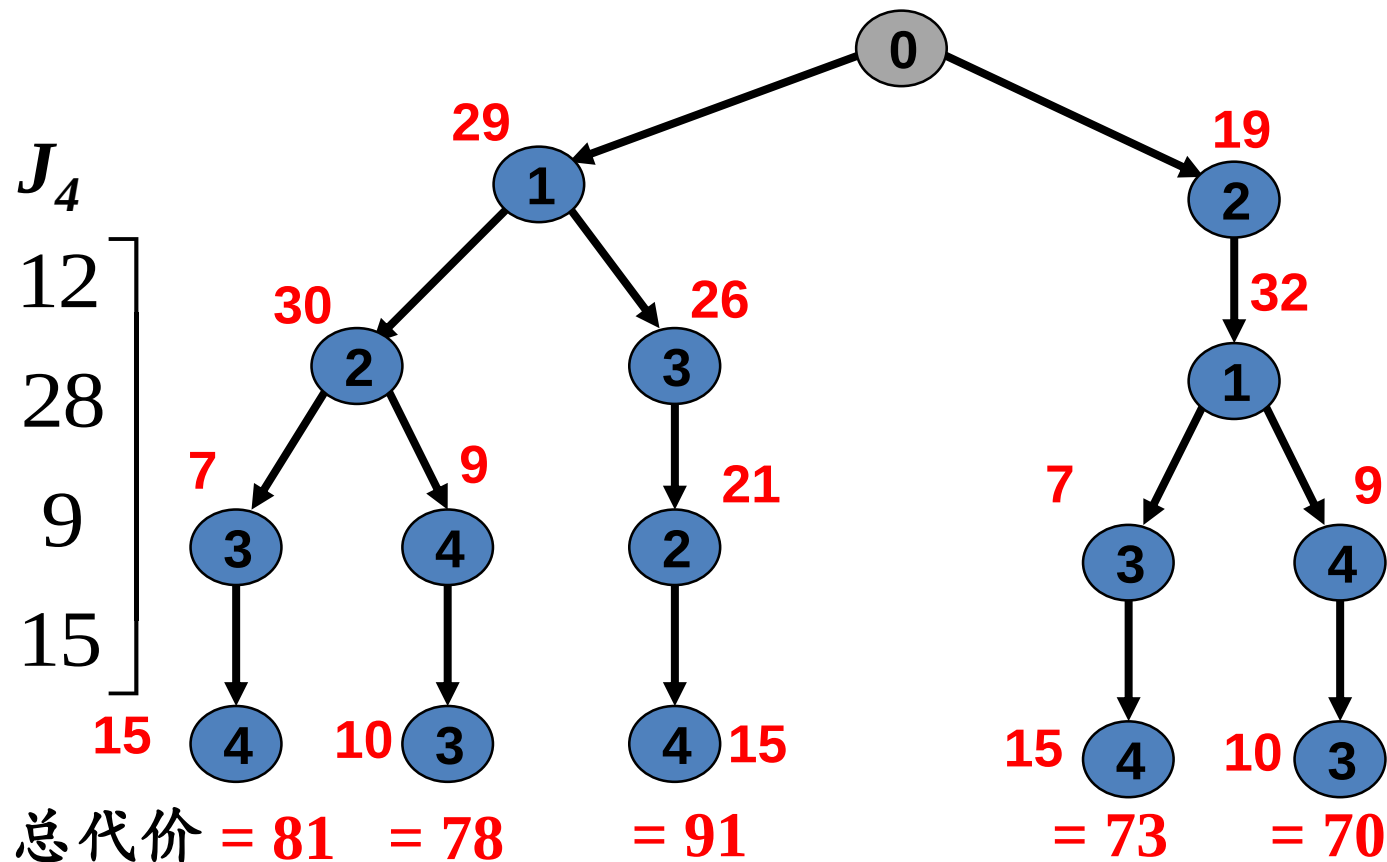
输入: 偏序集合 S , 树根root。

输出: 由 S 的所有拓朴排序序列构成的树。

1. 生成树根root;
2. 选择偏序集中没有前序元素的所有元素, 作为root的子结点;
3. **For** root的每个子结点 v **Do**
4. $S = S - \{v\}$;
5. 把 v 作为根, 递归地处理 S 。

代价矩阵

	J_1	J_2	J_3	J_4
P_1	29	19	17	12
P_2	32	30	26	28
P_3	3	21	7	9
P_4	18	13	10	15



人员安排问题的所有可行解都已求出（穷举所有的可能解）

有没有更有效的算法？

求解问题的分支界限搜索算法

- 计算解的代价的下界

命题2. 把代价矩阵某行（列）的各元素减去同一个数，
不影响优化解的求解。

- 代价矩阵的每行（列）减去同一个数（该行或列的最小数），使得每行和每列至少有一个零，其余各元素非负。
- 每行（列）减去的数的和即为解的下界。

求解问题的分支界限搜索算法

例.

- 代价矩阵的每行（列）减去同一个数（该行或列的最小数），使得每行和每列至少有一个零，其余各元素非负。
- 每行（列）减去的数的和即为解的下界。

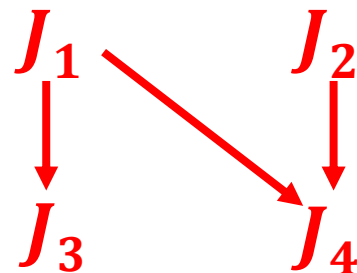
$$\begin{array}{c}
 \begin{array}{c} J_1 \quad J_2 \quad J_3 \quad J_4 \\
 \begin{array}{l} P_1 \\ P_2 \\ P_3 \\ P_4 \end{array} \left[\begin{array}{cccc} 29 & 19 & 17 & 12 \\ 32 & 30 & 26 & 28 \\ 3 & 21 & 7 & 9 \\ 18 & 13 & 10 & 15 \end{array} \right] \begin{array}{l} -12 \\ -26 \\ -3 \\ -10 \end{array}
 \end{array}
 \end{array}
 \xrightarrow{\text{Red Arrow}}
 \begin{array}{c}
 \begin{array}{c} J_1 \quad J_2 \quad J_3 \quad J_4 \\
 \left[\begin{array}{cccc} 17 & 7 & 5 & 0 \\ 6 & 4 & 0 & 2 \\ 0 & 18 & 4 & 6 \\ 8 & 3 & 0 & 5 \end{array} \right] \begin{array}{l} \\ \\ \\ -3 \end{array}
 \end{array}
 \end{array}
 \xrightarrow{\text{Red Arrow}}
 \begin{array}{c}
 \begin{array}{c} J_1 \quad J_2 \quad J_3 \quad J_4 \\
 \left[\begin{array}{cccc} 17 & 4 & 5 & 0 \\ 6 & 1 & 0 & 2 \\ 0 & 15 & 4 & 6 \\ 8 & 0 & 0 & 5 \end{array} \right]
 \end{array}
 \end{array}$$

得到一个具有最小代价 $12 + 26 + 3 + 10 + 3 = 54$ 的任务分配方案：

$$P_1 \rightarrow J_4, \quad P_2 \rightarrow J_3, \quad P_3 \rightarrow J_1, \quad P_4 \rightarrow J_2$$

它不满足偏序约束，故不是可行解

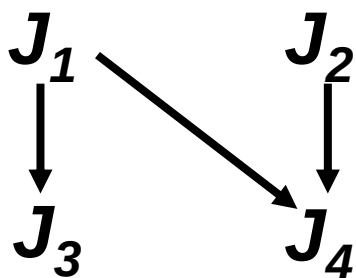
解代价下界 54 有没有用处？如何用？



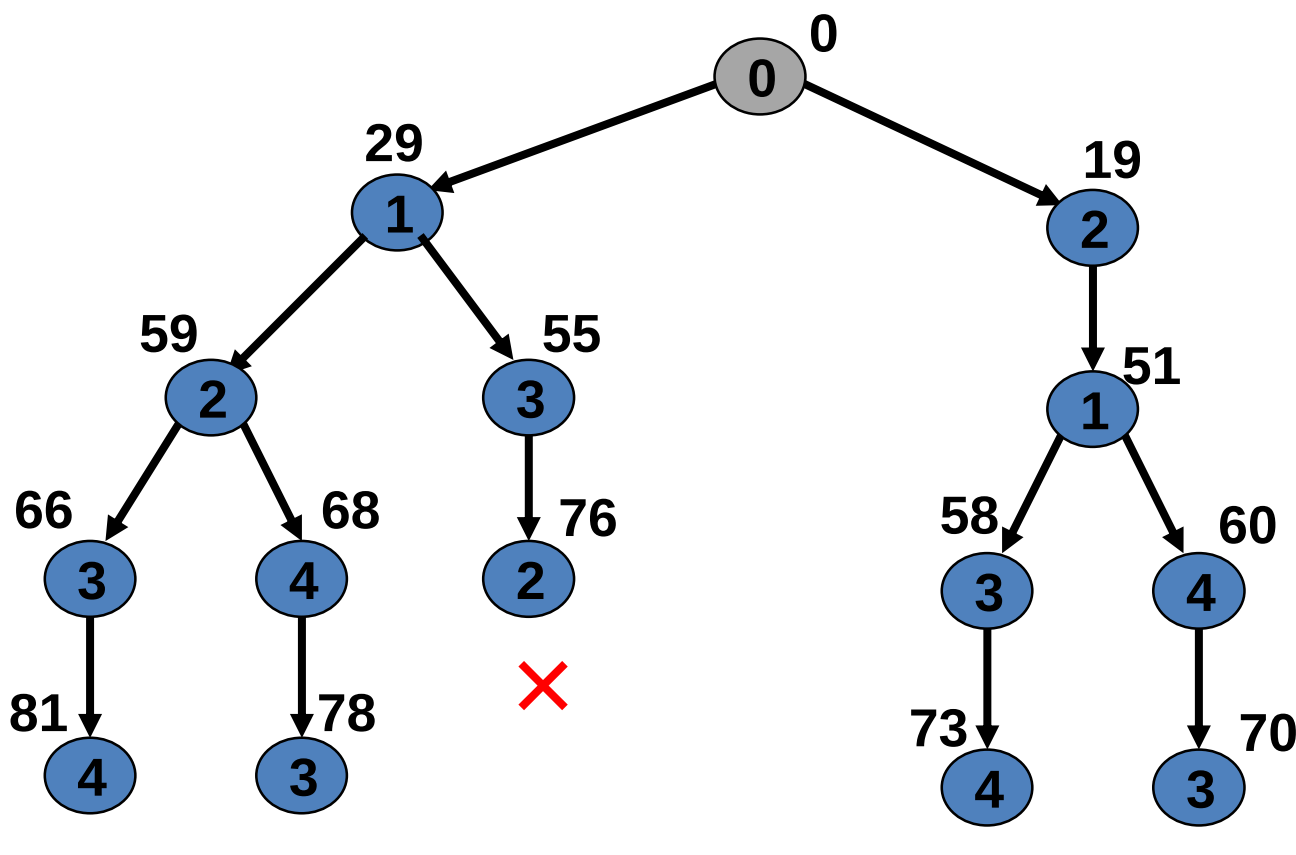
用原始代价矩阵进行分支界限求解

原始代价矩阵

29	19	17	12
32	30	26	28
3	21	7	9
18	13	10	15



启发测度函数：路径和



被分配到的人

1

2

3

4

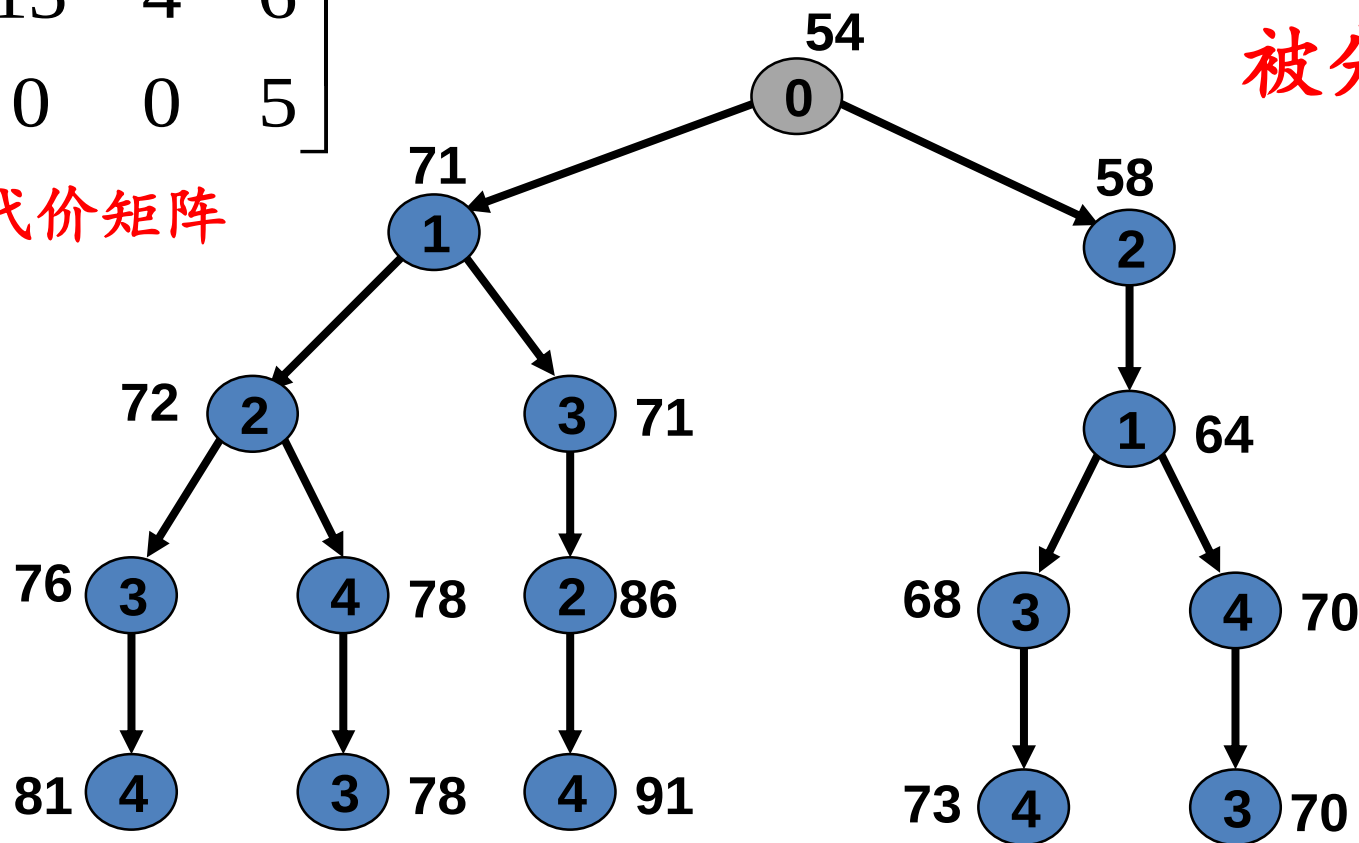
原始代价矩阵无法进行高效的剪枝

求解问题的分支界限搜索算法

- 解空间的加权树表示

$$\begin{bmatrix} 17 & 4 & 5 & 0 \\ 6 & 1 & 0 & 2 \\ 0 & 15 & 4 & 6 \\ 8 & 0 & 0 & 5 \end{bmatrix}$$

优化代价矩阵



被分配到的人

1

2

3

4

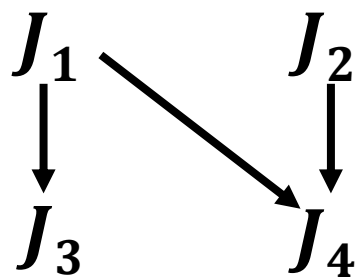
- 分支界限搜索（使用爬山法）算法

1. 建立根结点，其权值为解代价下界；
2. 使用爬山法，类似于拓朴排序序列树生成算法求解问题，每产生一个结点，其权值为加工后的代价矩阵对应元素加其父结点权值；
3. 一旦发现一个可能解，将其代价作为界限，循环地进行分支界限搜索：剪掉不能导致优化解的解，使用爬山法继续扩展新增结点，直至发现优化解。

求解问题的分支界限搜索算法

• 例

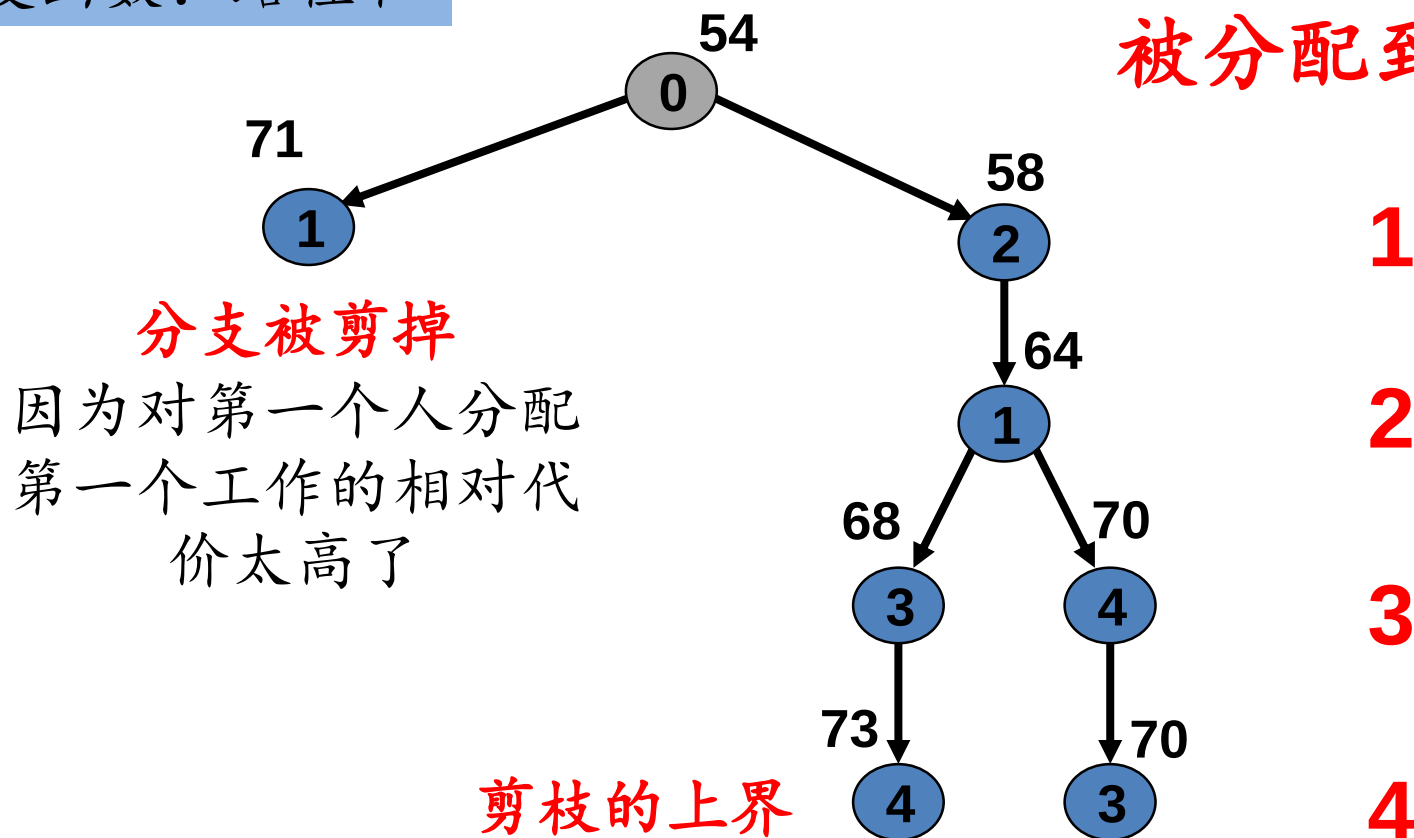
$\{P_1, P_2, P_3, P_4\}$



17	4	5	0
6	1	0	2
0	15	4	6
8	0	0	5

启发测度函数：路径和

被分配到的人



求解问题的分支界限搜索算法

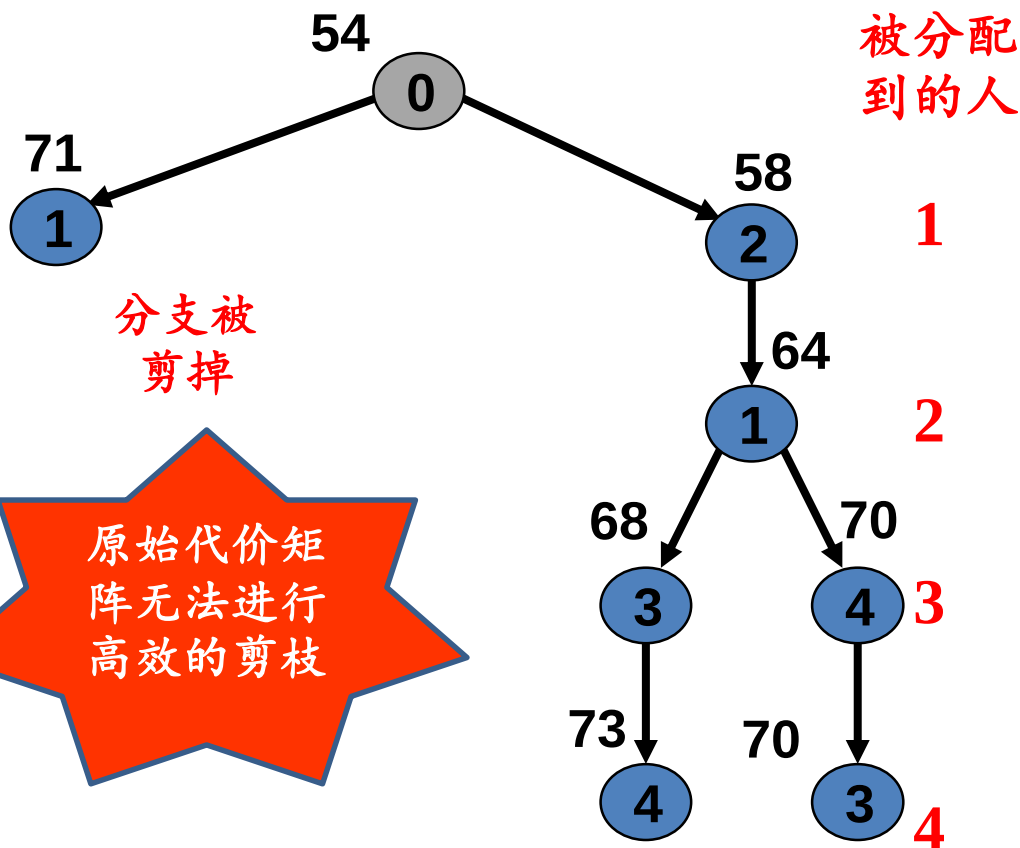
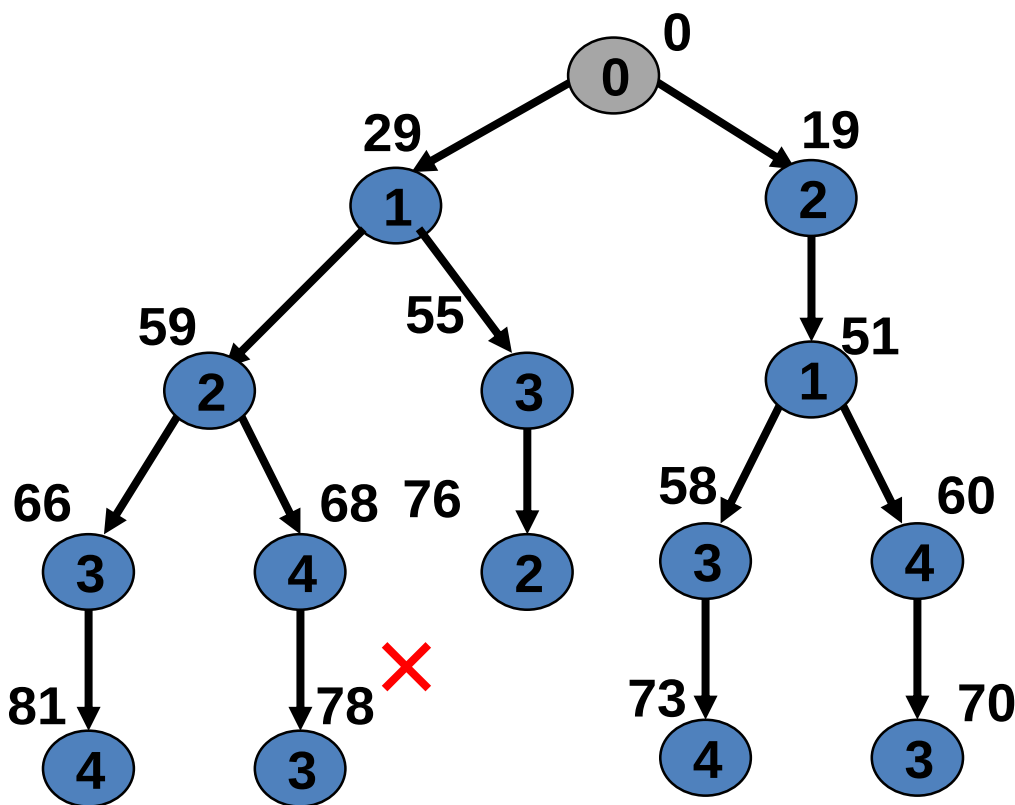
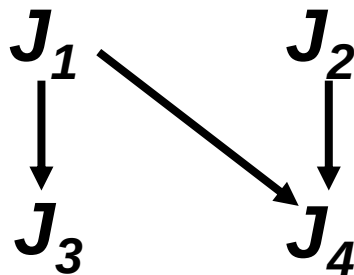
原始代价矩阵

29	19	17	12
32	30	26	28
3	21	7	9
18	13	10	15

优化代价矩阵

17	4	5	0
6	1	0	2
0	15	4	6
8	0	0	5

$\{P_1, P_2, P_3, P_4\}$



原始代价矩阵无法进行高效的剪枝

本讲内容

11.1 暴力美学：搜索漫谈

11.2 深度优先与广度优先

11.3 搜索的优化

11.4 剪枝方法论与人员安排问题

11.5 旅行商问题

11.6 A*算法

问题的定义

输入：连通图 $G = (V, E)$ ，每个结点都没有到自身的边，每对结点之间都有一条非负加权边。

输出：一条由任意一个结点开始，经过每个结点一次，最后返回开始结点的路径，该路径的代价（即权值之和）最小。

应用：车辆调度问题、计算机电路板布线问题、单个机器上的工序调度问题等等。



转换为树搜索问题

- 所有解集合作为树根，其权值由代价矩阵使用上节方法计算；
- 用爬山法递归地划分解空间，得到二叉树；
- 划分过程：
 - 选定一条边 (i,j)
 - 所有包含边 (i,j) 的解集合作为左子树
 - 所有不包含边 (i,j) 的解集合作为右子树
 - 计算出左右子树的代价下界
 - 边 (i,j) 的选择原则是，在优化后的代价矩阵中 (i,j) 为0，并使右子树代价下界直接增加值最大

分支界限搜索算法

- 在上述二叉树建立算法中增加如下策略:
 - 发现优化解的上界 α ;
 - 如果一个子结点的代价下界超过 α , 则终止该结点的扩展。
- 下边我们用一个例子来说明算法

- 构造根结点，设代价矩阵如下

$j =$	1	2	3	4	5	6	7	
$i=1$	∞	3	93	13	33	9	57	-3
2	4	∞	77	42	21	16	34	-4
3	45	17	∞	36	16	28	25	-16
4	39	90	80	∞	56	7	91	-7
5	28	46	88	33	∞	25	57	-25
6	3	88	18	46	92	∞	7	-3
7	44	26	33	27	84	39	∞	-26
		-7	-1				-4	

- 根结点为所有解的集合
- 计算根结点的代价下界

- 得到如下根结点及其代价下界

所有解的集合

$L.B=96$

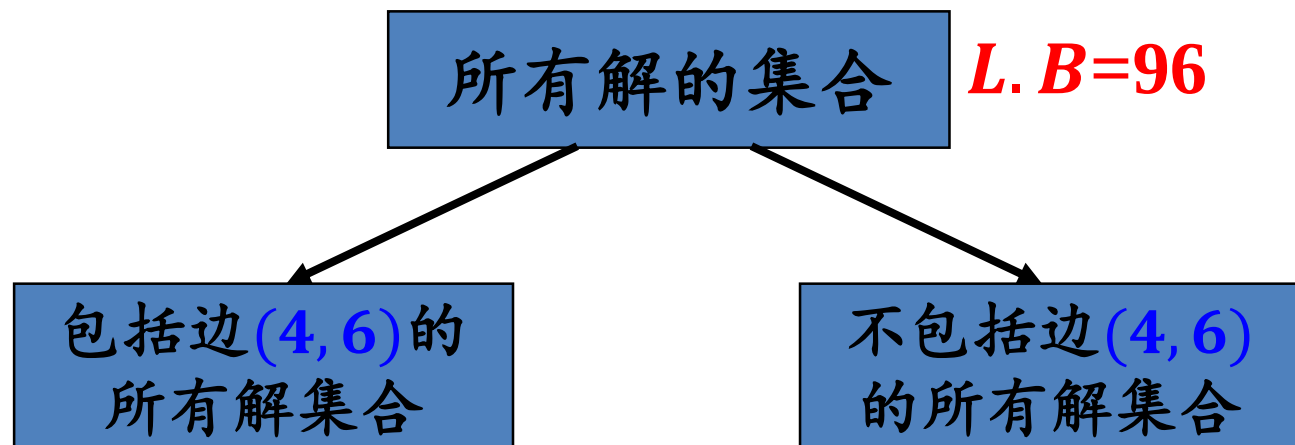
- 变换后的代价矩阵为

$j =$	1	2	3	4	5	6	7
$i=1$	∞	0	83	9	30	6	50
2	0	∞	66	37	17	12	26
3	29	1	∞	19	0	12	5
4	32	83	66	∞	49	0	80
5	3	21	56	7	∞	0	28
6	0	85	8	42	89	∞	0
7	18	0	0	0	58	13	∞

- 构造根结点的两个子结点

- 选择在优化后的代价矩阵中权值为0，并使右子结点代价下界的直接增加值最大的划分边(4,6)
- 建立根结点的子结点：
 - 左子结点为包括边(4,6)的所有解集合
 - 右子结点为不包括边(4,6)的所有解集合

∞	0	83	9	30	6	50
0	∞	66	37	17	12	26
29	1	∞	19	0	12	5
32	83	66	∞	49	0	80
3	21	56	7	∞	0	28
0	85	8	42	89	∞	0
18	0	0	0	58	13	∞

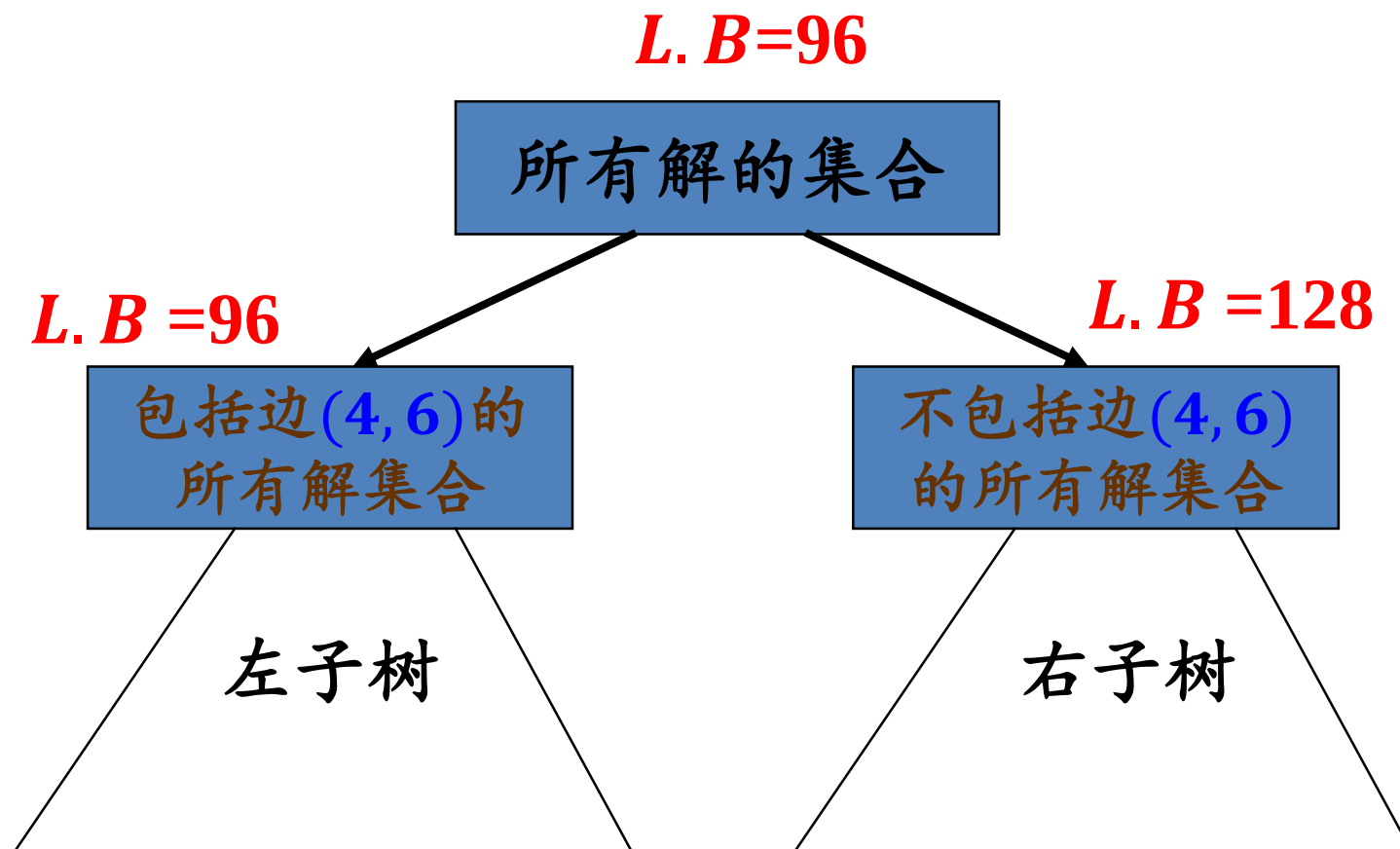


- 计算左右子结点的代价下界
- $(4, 6)$ 的代价为0，所以左结点代价下界**没有直接的增加**，仍为96。
- 我们来计算右结点的代价下界：
 - 如果一个解不包含 $(4, 6)$ ，它必包含**一条从4出发的边和进入结点6的边**。
 - 由变换后的代价矩阵可知，具有最小代价由4出发的边为 $(4, 1)$ ，代价为32。
 - 由变换后的代价矩阵可知，具有最小代价进入6的边为 $(5, 6)$ ，代价为0。
 - 因此，不包含 $(4, 6)$ 导致右结点代价**下界直接增加**： $96 + 32 + 0 = 128$ 。

∞	0	83	9	30	6	50
0	∞	66	37	17	12	26
29	1	∞	19	0	12	5
32	83	66	∞	49	0	80
3	21	56	7	∞	0	28
0	85	8	42	89	∞	0
18	0	0	0	58	13	∞

- 边 $(1, 2)$ 右子树增加代价： $6 + 0 = 6$
- 边 $(2, 1)$ 右子树增加代价： $12 + 0 = 12$
- 边 $(3, 5)$ 右子树增加代价： $1 + 17 = 18$
- 边 $(4, 6)$ 右子树增加代价： $32 + 0 = 32$
- 边 $(5, 6)$ 右子树增加代价： $3 + 0 = 3$
- 边 $(6, 1)$ 右子树增加代价： $0 + 0 = 0$
- 边 $(6, 7)$ 右子树增加代价： $0 + 5 = 5$
- 边 $(7, 2)$ 右子树增加代价： $0 + 0 = 0$
- 边 $(7, 3)$ 右子树增加代价： $0 + 8 = 8$
- 边 $(7, 4)$ 右子树增加代价： $0 + 7 = 7$

- 使用(4,6)划分后，仅考虑代价下界的直接增加，目前的树为：



递归地构造左右子树

- 构造左子树根对应的代价矩阵
 - 左子结点为包括边(4, 6)的所有解集合，所以矩阵的第4行和第6列应该被删除。
 - 由于边(4, 6)被使用，边(6, 4)不能再使用，所以代价矩阵的元素 $C(6, 4)$ 应该设置为 ∞ 。
 - 结果矩阵如下

$j =$	1	2	3	4	5	6	7
$i=1$	∞	0	83	9	30		50
2	0	∞	66	37	17		26
3	29	1	∞	19	0		5
4							
5	3	21	56	7	∞		28
6	0	85	8	∞	89		0
7	18	0	0	0	58		∞

递归地构造左右子树

- 使用 (4,6) 划分后，左子树根的代价下界没有直接增加，但带来了间接的增加
 - 矩阵的第5行不包含0
 - 第5行元素减3，左子树根代价下界间接增加了3为: $96 + 3 = 99$
 - 结果矩阵如下

j	1	2	3	4	5	6	7
$i=1$	∞	0	83	9	30		50
2	0	∞	66	37	17		26
3	29	1	∞	19	0		5
4							
5	0	18	53	4	∞		25 ← - 3
6	0	85	8	∞	89		0
7	18	0	0	0	58		∞

递归地构造左右子树

- 构造右子树根对应的代价矩阵
 - 右子结点为不包括边(4,6)的所有解集合，只需要把 $C(4,6)$ 设置为 ∞
 - 结果矩阵如下

$j =$	1	2	3	4	5	6	7
$i=1$	∞	0	83	9	30	6	50
2	0	∞	66	37	17	12	26
3	29	1	∞	19	0	12	5
4	32	83	66	∞	49	∞	80
5	3	21	56	7	∞	0	28
6	0	85	8	42	89	∞	0
7	18	0	0	0	58	13	∞

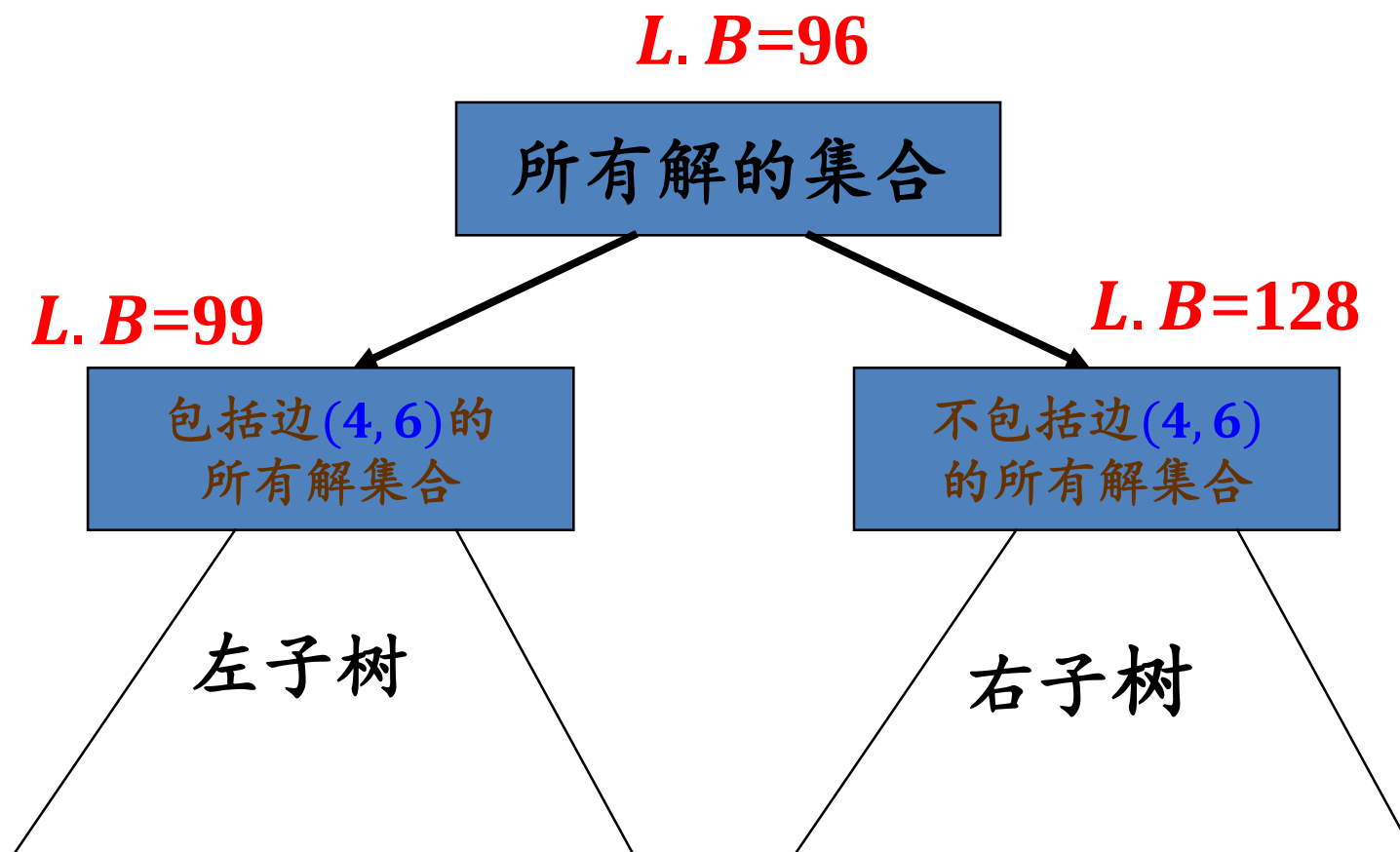
- 不包含边(4, 6)的右子树根的代价下界产生了**直接增加**，具体结算方式为：

- 矩阵的第4行不包含0
- 第4行元素减32
- 结果矩阵如下

	$j = 1$	2	3	4	5	6	7
$i = 1$	∞	0	83	9	30	6	50
2	0	∞	66	37	17	12	26
3	29	1	∞	19	0	12	5
4	0	51	34	∞	17	∞	48
5	3	21	56	7	∞	0	28
6	0	85	8	42	89	∞	0
7	18	0	0	0	58	13	∞

← - 32

- 目前的树为



- 使用爬山策略扩展左子树根
 - 选择边使右子结点直接代价下界增加最大的划分边(3,5)
 - 左子结点为包括边(3,5)的所有解集合
 - 右子结点为不包括边(3,5)的所有解集合
 - 计算左、右子结点的代价下界：99和117
- 目前树扩展为：

$j =$	1	2	3	4	5	6	7
$i=1$	∞	0	83	9	30		50
2	0	∞	66	37	17		26
3	29	1	∞	19	0		5
4							
5	0	18	53	4	∞		25
6	0	85	8	∞	89		0
7	18	0	0	0	58		∞

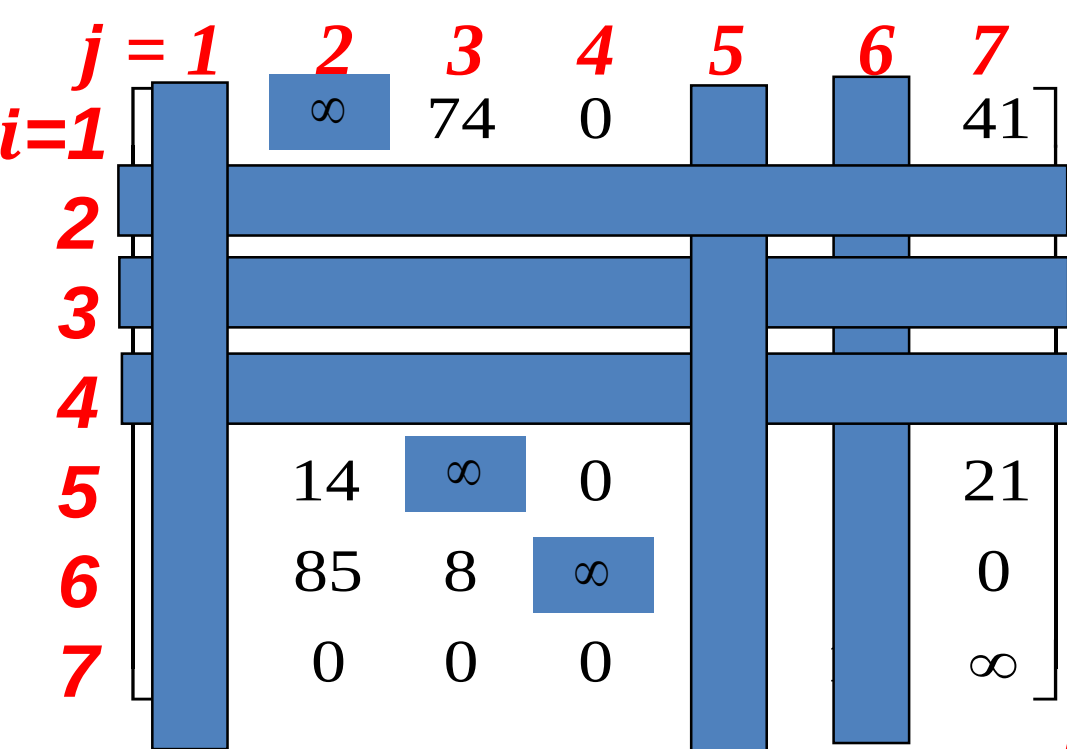
- 边(1,2)右子树增加代价： $9 + 0 = 9$
- 边(2,1)右子树增加代价： $17 + 0 = 17$
- 边(3,5)右子树增加代价： $1 + 17 = 18$
- 边(5,1)右子树增加代价： $4 + 0 = 4$
- 边(6,1)右子树增加代价： $0 + 0 = 0$
- 边(6,7)右子树增加代价： $0 + 5 = 5$
- 边(7,2)右子树增加代价： $0 + 0 = 0$
- 边(7,3)右子树增加代价： $0 + 8 = 8$
- 边(7,4)右子树增加代价： $0 + 4 = 4$

$j =$	1	2	3	4	5	6	7
$i=1$	∞	0	83	9			50
2	0	∞	66	37			26
3							
4							
5	0	18	∞	4			25
6	0	85	8	∞			0
7	18	0	0	0			∞

边(1,2)右子树增加代价: $9 + 0 = 9$
 边(2,1)右子树增加代价: $26 + 0 = 26$
 边(5,1)右子树增加代价: $4 + 0 = 4$
 边(6,1)右子树增加代价: $0 + 0 = 0$
 边(6,7)右子树增加代价: $0 + 25 = 25$
 边(7,2)右子树增加代价: $0 + 0 = 0$
 边(7,3)右子树增加代价: $0 + 8 = 8$
 边(7,4)右子树增加代价: $0 + 4 = 4$

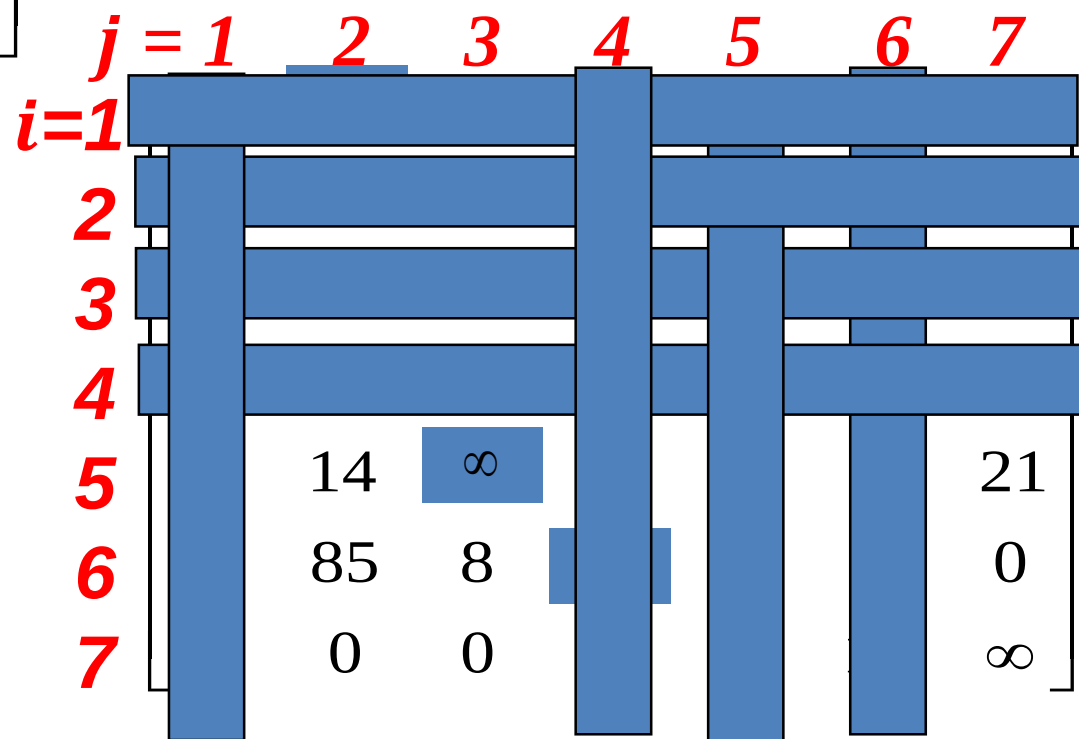
第1行减去9, 第5行减去4,
 间接增加 $=9+4=13$

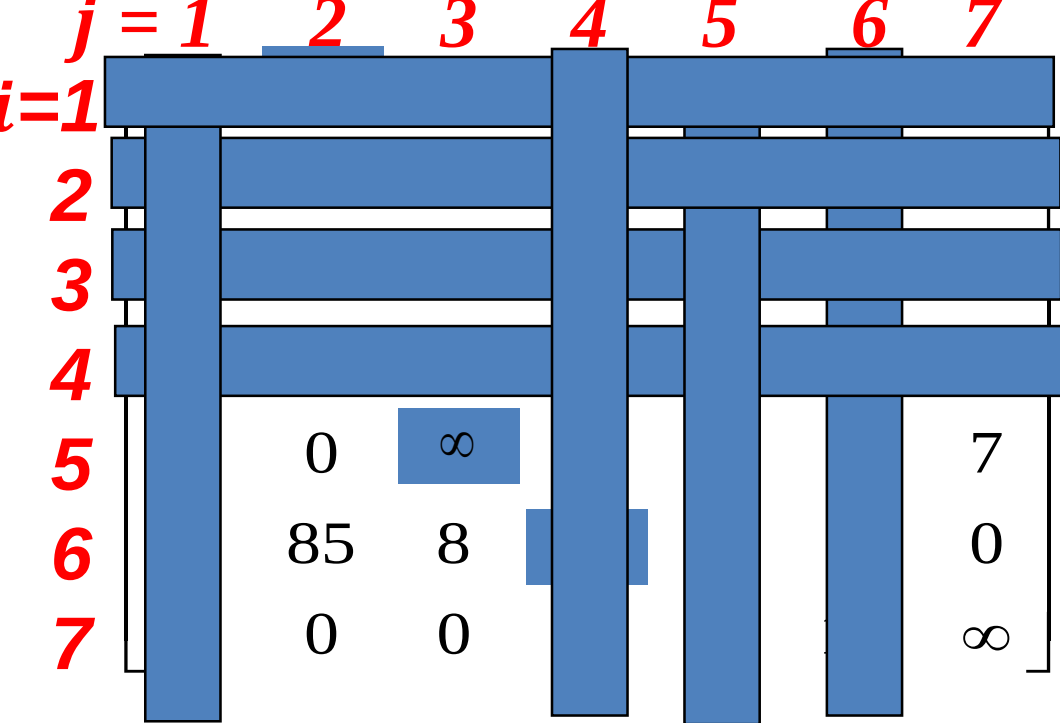
$j =$	1	2	3	4	5	6	7
$i=1$		∞	83	9			50
2							
3							
4							
5	18	∞	4				25
6	85	8	∞				0
7	0	0	0				∞



边(1,4)右子树增加代价: $41 + 0 = 41$
 边(5,4)右子树增加代价: $14 + 0 = 14$
 边(6,7)右子树增加代价: $8 + 21 = 29$
 边(7,2)右子树增加代价: $14 + 0 = 14$
 边(7,3)右子树增加代价: $0 + 8 = 8$
 边(7,4)右子树增加代价: $0 + 0 = 0$

第5行减去14,
间接增加14



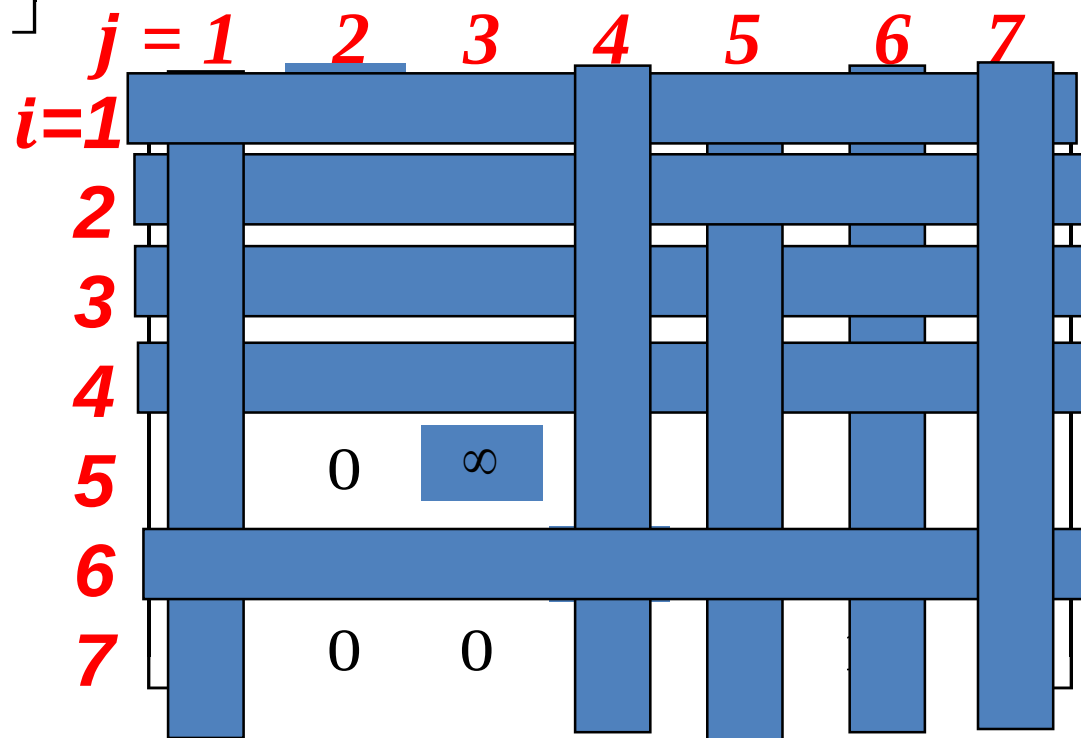


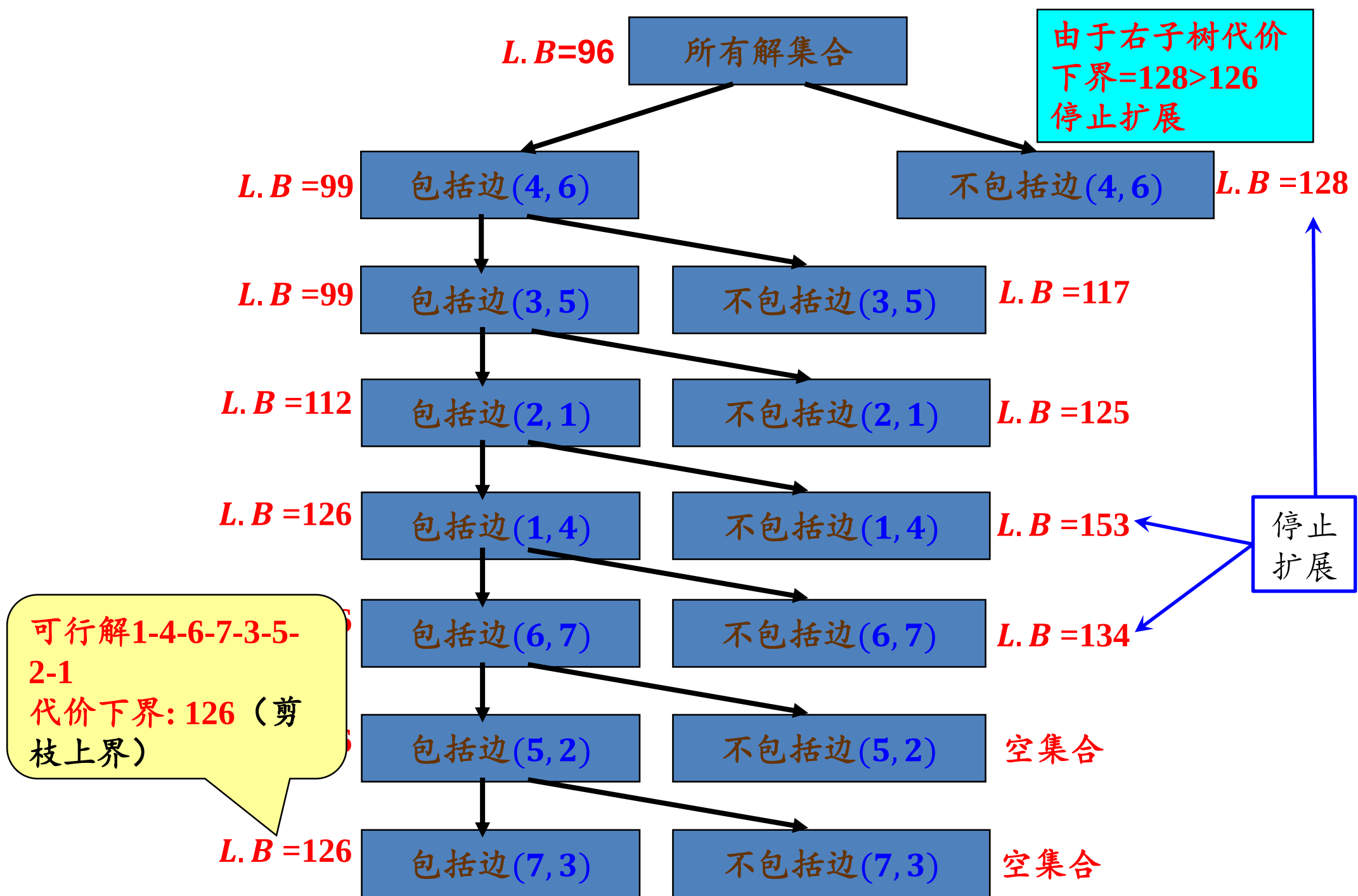
边(5,2)右子树增加代价: $7 + 0 = 7$

边(6,7)右子树增加代价: $8 + 7 = 15$

边(7,2)右子树增加代价: $0 + 0 = 0$

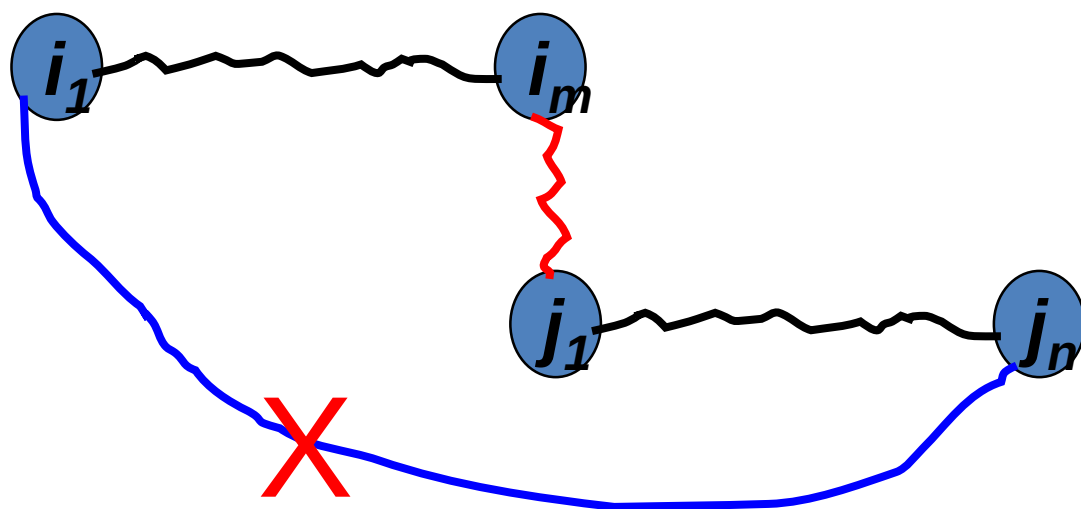
边(7,3)右子树增加代价: $0 + 8 = 8$





注意:

如果 $i_1 - i_2 - \dots - i_m$ 和 $j_1 - j_2 - \dots - j_n$ 已被包含在一个正在构造的路径中, (i_m, j_1) 被加入, 则必须避免 j_n 到 i_1 的路径被加入。否则出现环。



算法概要

- 1、树根为所有解集合；
- 2、使用代价矩阵，计算根结点表示的解集合的代价下界；
- 3、使用爬山策略扩展当前的树结点 α 直至找到一个解：
 - 选择满足下列条件的边 (x, y) ：
 - 使右子树代价下界增加最大
 - 使左子树代价下界不变
 - 构造 α 的左右子树：
 - 左子树为包括边 (x, y) 的所有解集合；
 - 右子树为不包括边 (x, y) 的所有解集合；
 - 计算左右子树解代价下界；
 - 构造左右子树的代价矩阵，修改其解代价下界；
- 4、用找到解的代价进行剪枝爬山搜索，直到找到最优解。

本讲内容

11.1 暴力美学：搜索漫谈

11.2 深度优先与广度优先

11.3 搜索的优化

11.4 剪枝方法论与人员安排问题

11.6 旅行商问题

11.6 A*算法

A*算法的基本思想

- A*算法与分支界限策略的比较
 - 分支界限策略是为了剪掉不能达到优化解的分支
 - 分支界限策略的关键是“界限”
 - A*算法的核心是告诉我们在某些情况下，我们得到的解一定是优化解，于是算法可以停止
 - A*算法试图尽早地发现优化解
 - A*算法通常使用Best-first策略求解优化问题

A*算法的基本思想

- A*算法关键：代价函数

- 对于任意结点 n

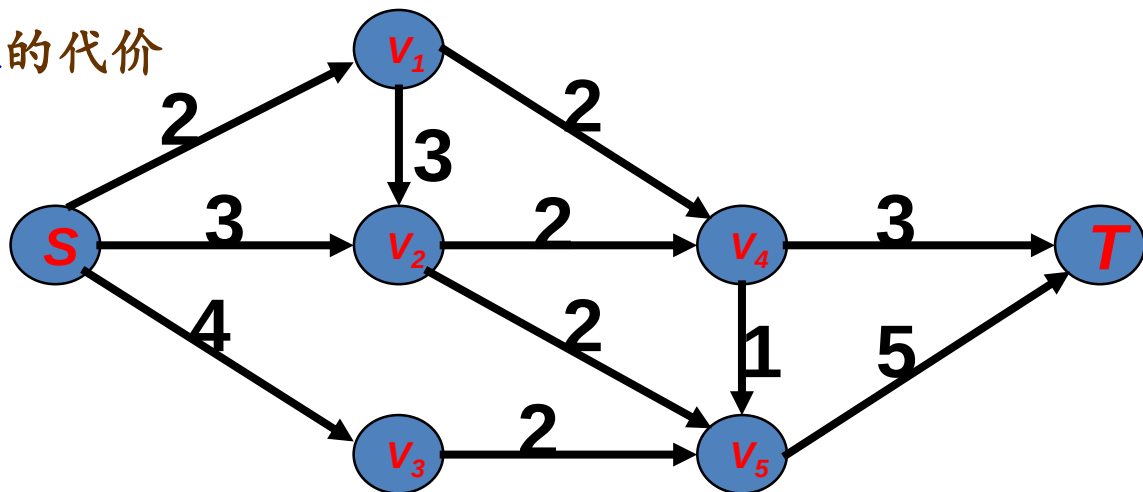
- $g(n)$ = 从树根（起始结点）到 n 的代价
- $h^*(n)$ = 从 n 结点到目标结点的优化路径的代价
- $f^*(n) = g(n) + h^*(n)$ 是结点 n 的代价

- $h^*(n)$ 的值是多少？

- 不知道！
- 于是 $f^*(n)$ 也不知道

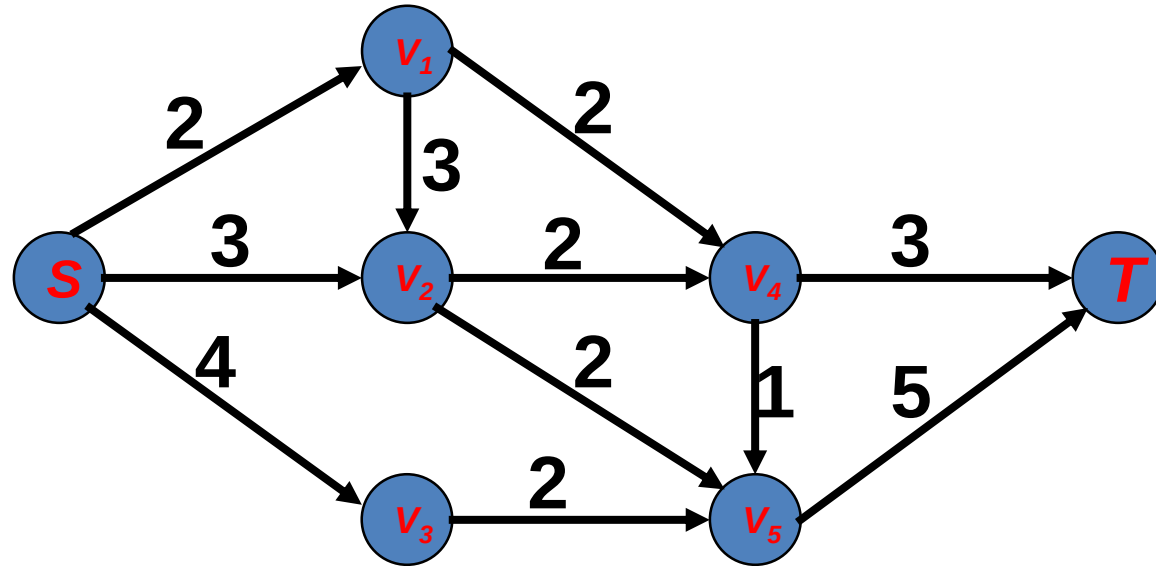
- 估计 $h^*(n)$

- 使用任何方法去估计 $h^*(n)$ ，用 $h(n)$ 表示 $h^*(n)$ 的估计
- $h(n) \leq h^*(n)$ 总为真
- $f(n) = g(n) + h(n) \leq g(n) + h^*(n) = f^*(n)$ 定义为结点 n 的代价



最短路径问题

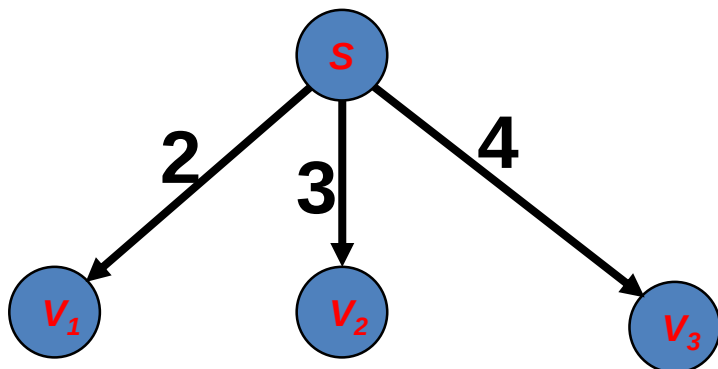
- 输入:



- 输出: 发现一个从 S 到 T 的最短路径

最短路径问题

- 求解树的第一阶段

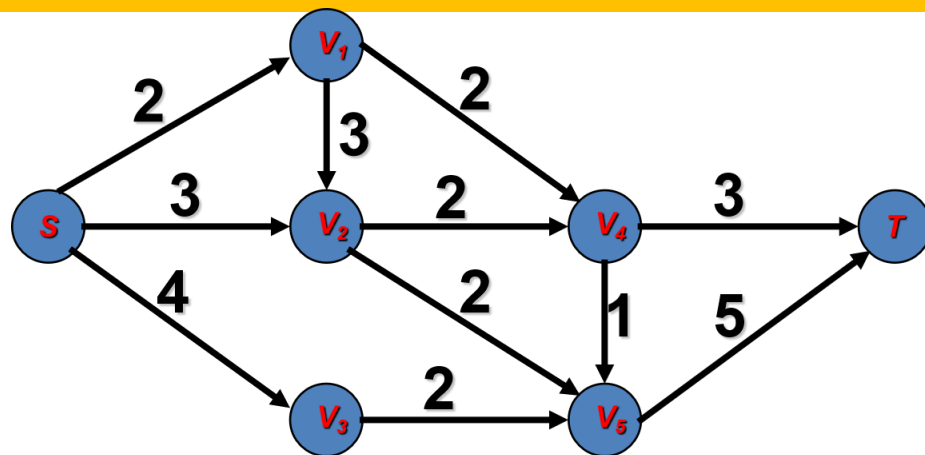


$$g(V_1) = 2, \quad g(V_2) = 3, \quad g(V_3) = 4$$

~~$$h^*(V_1) = 5, \quad f^*(V_1) = g(V_1) + h^*(V_1) = 7$$~~

对于任意结点 n

- $g(n)$ = 从树根到 n 的代价
- $h^*(n)$ = 从 n 到目标结点的优化路径的代价
- $f^*(n) = g(n) + h^*(n)$ 是结点 n 的代价



- 估计 $h^*(V_1)$

- 从 V_1 出发有两种可能：代价为2，代价为3，最小者为2
- $h^*(V_1) \geq 2$ ，选择 $h(V_1) = 2$ 为 $h^*(V_1)$ 的估计值
- $f(V_1) = g(V_1) + h(V_1) = 4$ 为 V_1 的代价

A*算法的规则

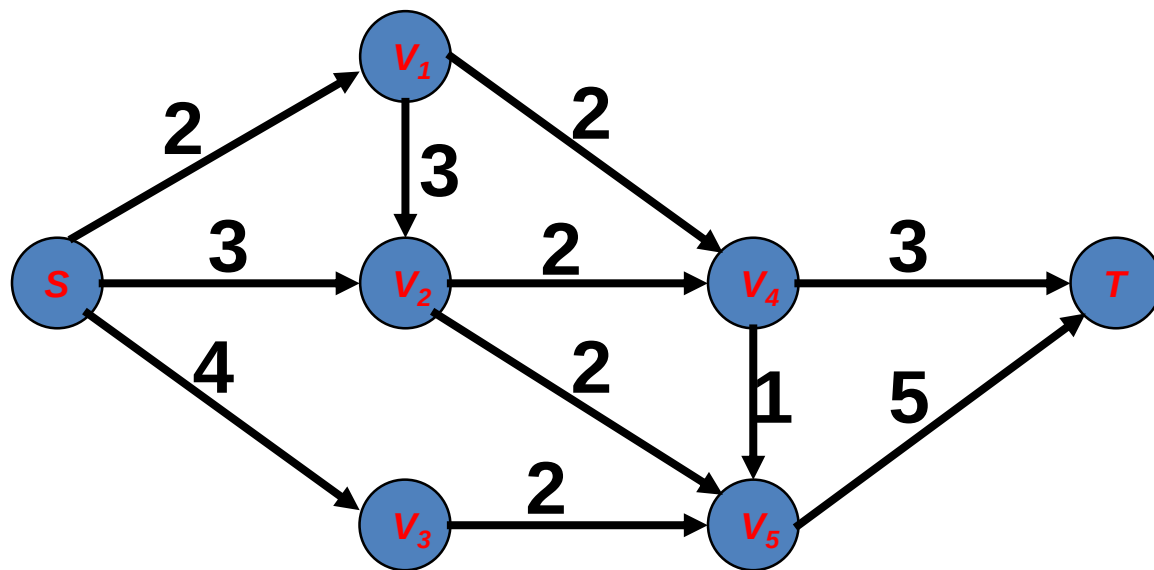
- 使用Best-first策略搜索树;
- 结点 n 的代价函数为 $f(n) = g(n) + h(n)$, $g(n)$ 是从根到 n 的路径代价, $h(n)$ 是从 n 到某个目标结点的优化路径代价;
- 对于所有 n , $h(n) \leq h^*(n)$;
- 当选择到的结点是目标结点时, 算法停止, 返回一个优化解。

对于任意结点 n

- $g(n)$ = 从树根到 n 的代价
- $h^*(n)$ = 从 n 到目标结点的优化路径的代价
- $f^*(n) = g(n) + h^*(n)$ 是结点 n 的代价

应用A*算法求解最短路径问题

- 问题的输入:



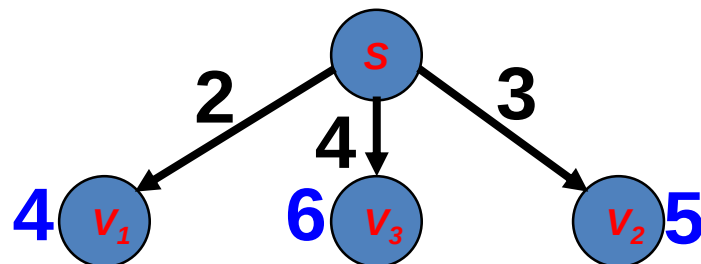
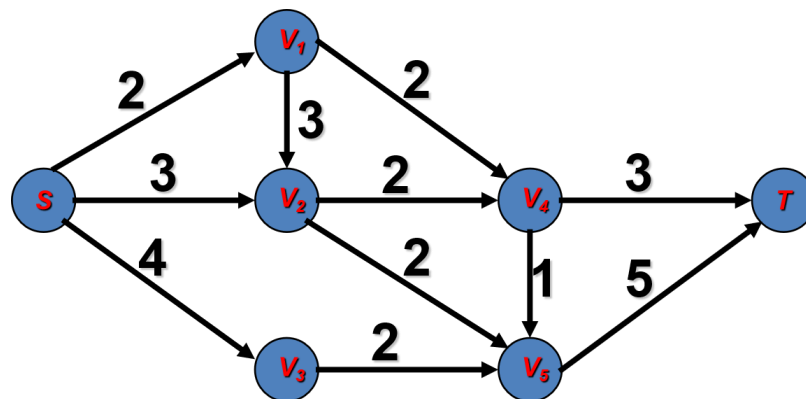
- A*算法的执行全过程

应用A*算法求解最短路径问题

Step 1

对于任意结点 n

- $g(n)$ = 从树根到 n 的代价
- $h^*(n)$ = 从 n 到目标结点的优化路径的代价
- $f^*(n) = g(n) + h^*(n)$ 是结点 n 的代价



$$g(V_1) = 2 \quad h(V_1) = \min\{2, 3\} = 2 \quad f(V_1) = 2 + 2 = 4$$

$$g(V_3) = 4 \quad h(V_3) = \min\{2\} = 2 \quad f(V_3) = 4 + 2 = 6$$

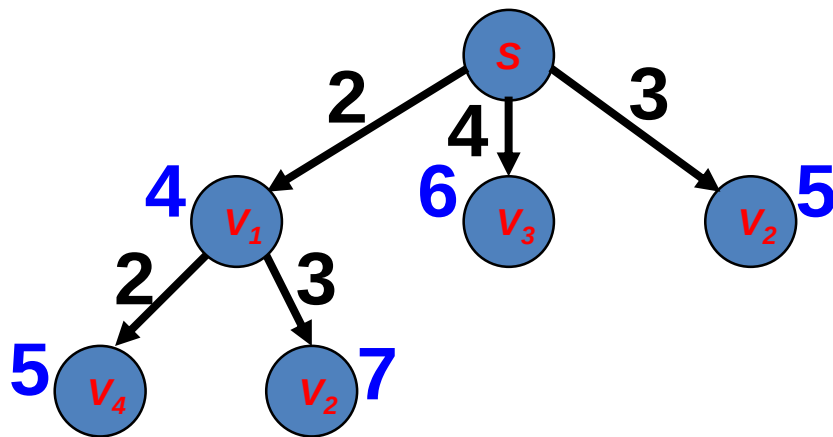
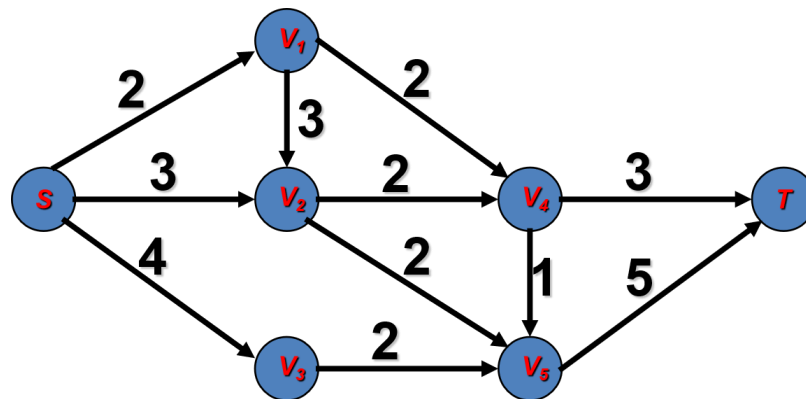
$$g(V_2) = 3 \quad h(V_2) = \min\{2, 2\} = 2 \quad f(V_2) = 2 + 2 = 5$$

应用A*算法求解最短路径问题

Step 2. 扩展 V_1

对于任意结点 n

- $g(n)$ = 从树根到 n 的代价
- $h^*(n)$ = 从 n 到目标结点的优化路径的代价
- $f^*(n) = g(n) + h^*(n)$ 是结点 n 的代价



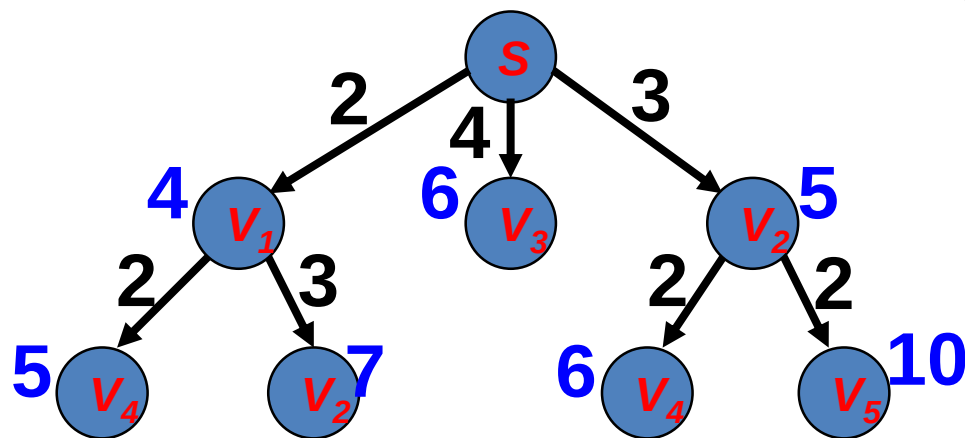
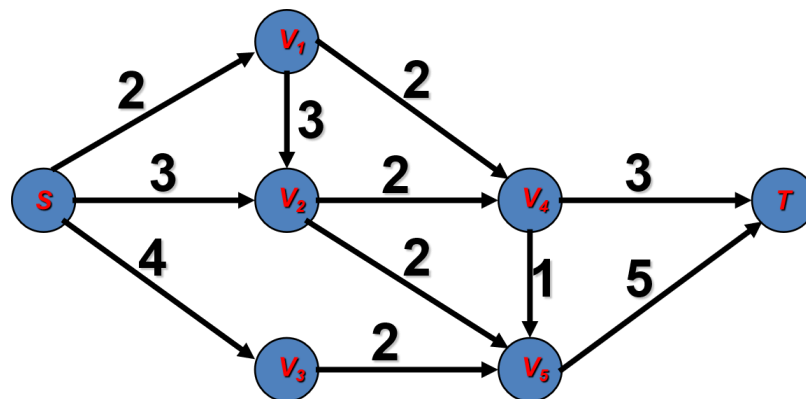
$$\begin{array}{lll} g(V_4) = 2 + 2 = 4 & h(V_4) = \min\{3, 1\} = 1 & f(V_4) = 4 + 1 = 5 \\ g(V_2) = 2 + 3 = 5 & h(V_2) = \min\{2, 2\} = 2 & f(V_2) = 5 + 2 = 7 \end{array}$$

应用A*算法求解最短路径问题

Step 3. 扩展 V_2

对于任意结点 n

- $g(n)$ = 从树根到 n 的代价
- $h^*(n)$ = 从 n 到目标结点的优化路径的代价
- $f^*(n) = g(n) + h^*(n)$ 是结点 n 的代价



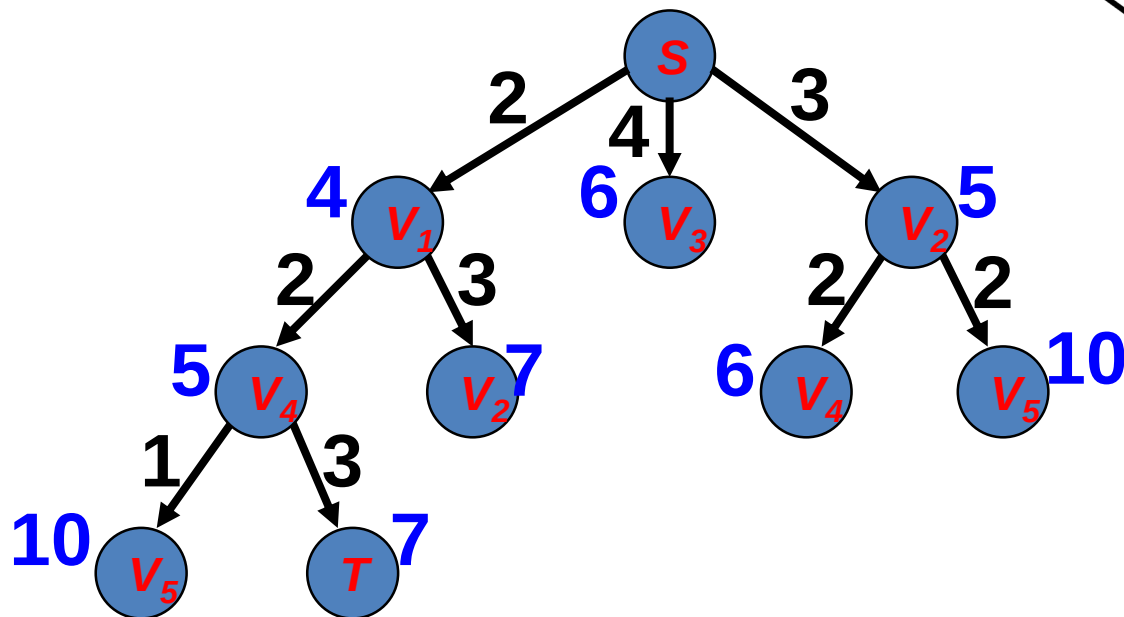
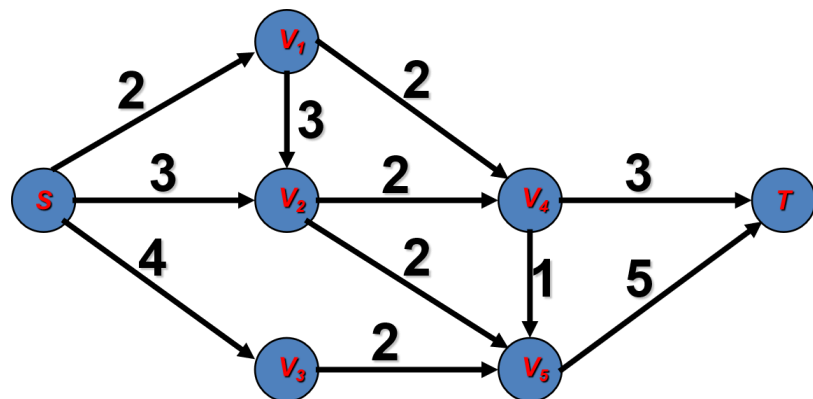
$$\begin{aligned} g(V_4) &= 3 + 2 = 5 & h(V_4) &= \min\{3, 1\} = 1 & f(V_4) &= 5 + 1 = 6 \\ g(V_5) &= 3 + 2 = 5 & h(V_5) &= \min\{5\} = 5 & f(V_5) &= 5 + 5 = 10 \end{aligned}$$

应用A*算法求解最短路径问题

Step 4. 扩展 V_4

对于任意结点 n

- $g(n)$ = 从树根到 n 的代价
- $h^*(n)$ = 从 n 到目标结点的优化路径的代价
- $f^*(n) = g(n) + h^*(n)$ 是结点 n 的代价



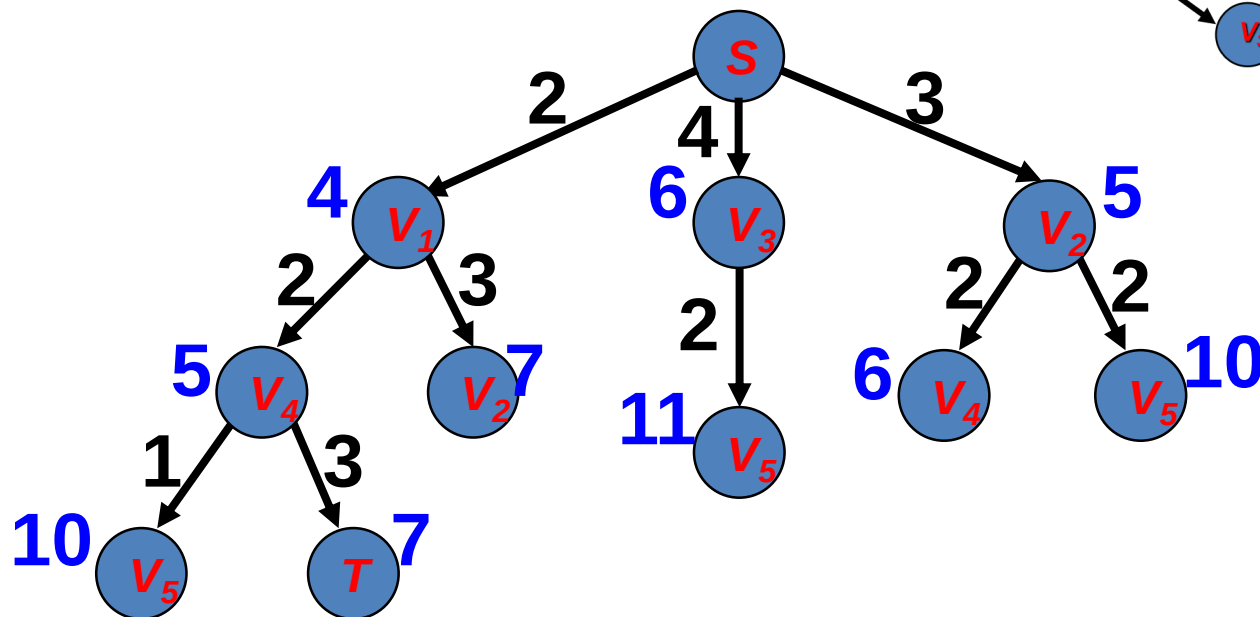
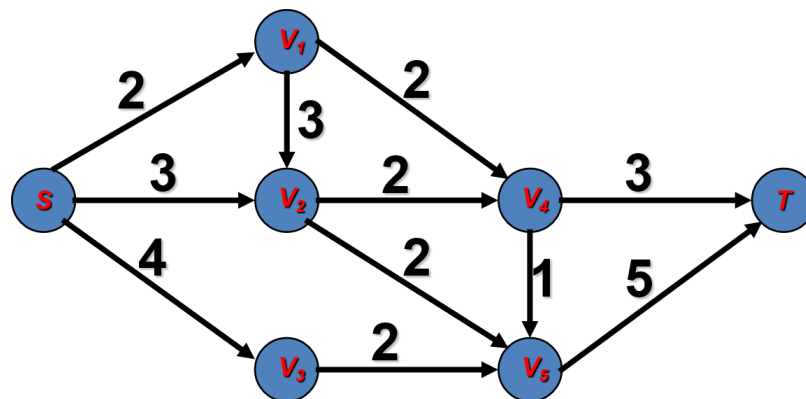
$$\begin{aligned} g(V_5) &= 2 + 2 + 1 = 5 & h(V_5) &= \min\{5\} = 5 & f(V_5) &= 5 + 5 = 10 \\ g(T) &= 2 + 2 + 3 = 7 & h(T) &= 0 & f(T) &= 7 + 0 = 7 \end{aligned}$$

应用A*算法求解最短路径问题

Step 5. 扩展 V_3

对于任意结点 n

- $g(n)$ = 从树根到 n 的代价
- $h^*(n)$ = 从 n 到目标结点的优化路径的代价
- $f^*(n) = g(n) + h^*(n)$ 是结点 n 的代价



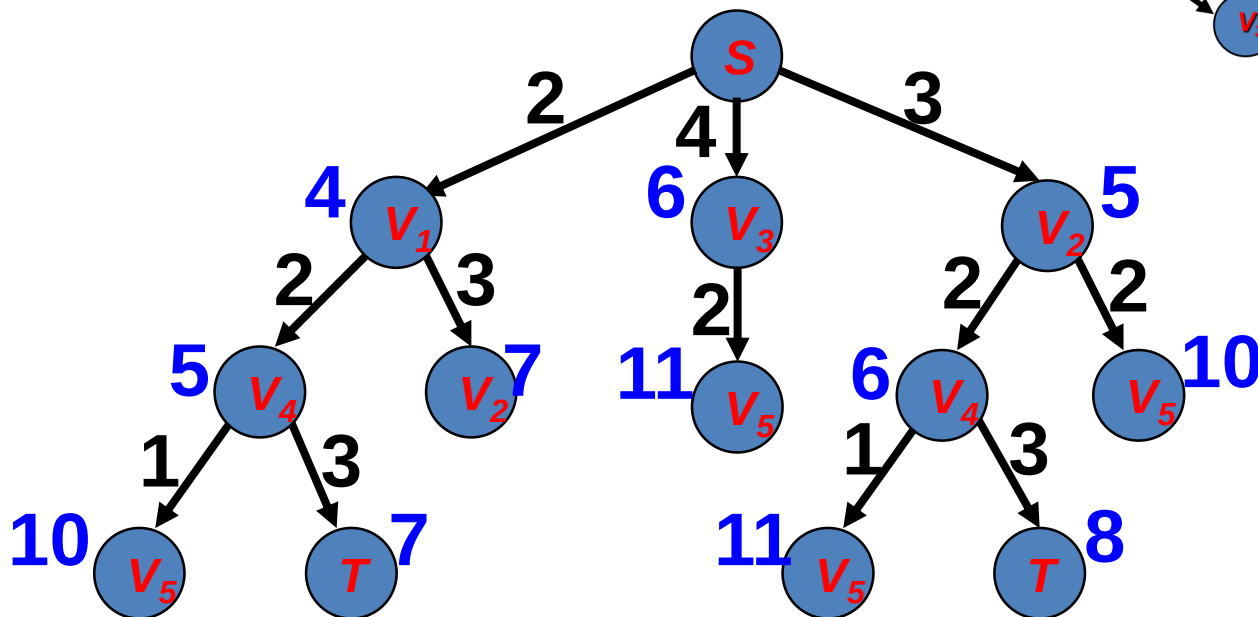
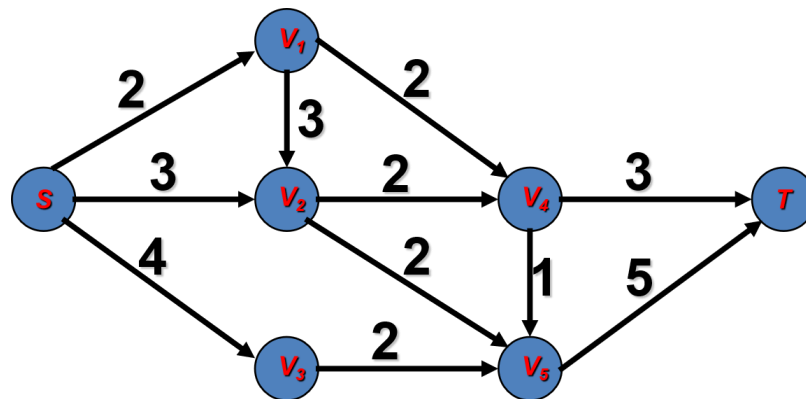
$$g(V_5) = 4 + 2 = 6 \quad h(V_5) = \min\{5\} = 5 \quad f(V_5) = 6 + 5 = 11$$

应用A*算法求解最短路径问题

Step 11. 扩展 V_4

对于任意结点 n

- $g(n)$ = 从树根到 n 的代价
- $h^*(n)$ = 从 n 到目标结点的优化路径的代价
- $f^*(n) = g(n) + h^*(n)$ 是结点 n 的代价



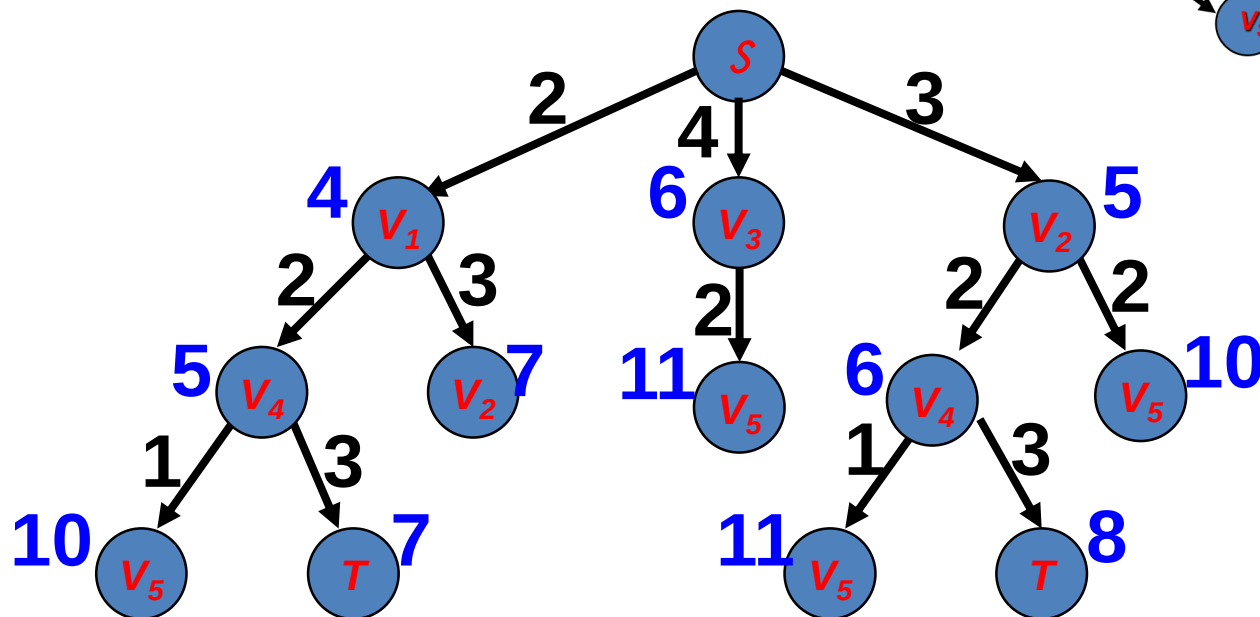
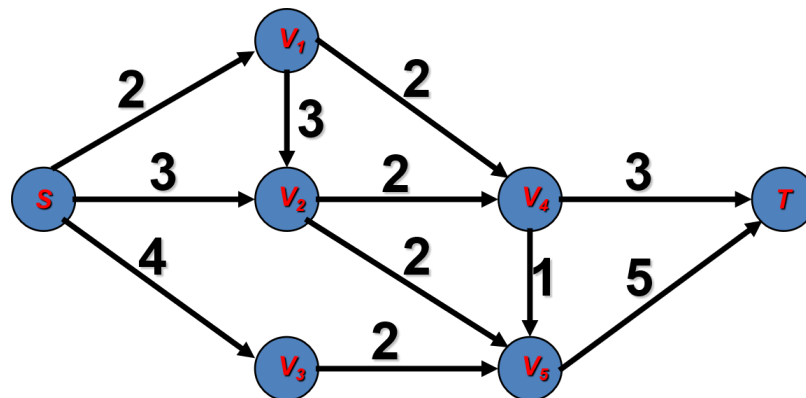
$$\begin{aligned} g(V_5) &= 3 + 2 + 1 = 6 & h(V_5) &= \min\{5\} = 5 & f(V_5) &= 6 + 5 = 11 \\ g(T) &= 3 + 2 + 3 = 8 & h(T) &= 0 & f(T) &= 8 + 0 = 8 \end{aligned}$$

应用A*算法求解最短路径问题

Step 7. 扩展T

对于任意结点 n

- $g(n)$ = 从树根到 n 的代价
- $h^*(n)$ = 从 n 到目标结点的优化路径的代价
- $f^*(n) = g(n) + h^*(n)$ 是结点 n 的代价



因为 T 是目标结点，所以我们得到解：

$S \rightarrow V_1 \rightarrow V_4 \rightarrow T$

A*算法本质的正确性

对于任意结点 n

- $g(n)$ = 从树根到 n 的代价
- $h^*(n)$ = 从 n 到目标结点的优化路径的代价
- $f^*(n) = g(n) + h^*(n)$ 是结点 n 的代价

定理1. 使用Best-first策略搜索树，如果A*选择的结点是目标结点，则该结点表示的解是优化解。

证明. 令 n 是任意扩展到的结点， t 是选中目标结点（可为阶段目标结点，可为最终目标结点）。

证明 $f(t) = g(t)$ （满足该等式， t 为最终目标结点）是优化解代价。

- (1) A*算法使用Best-first策略， $f(t) \leq f(n)$ 。
- (2) A*算法使用 $h(n) \leq h^*(n)$ 估计规则， $f(t) \leq f(n) \leq f^*(n)$ 。
- (3) $\{f^*(n)\}$ 中必有一个为优化解的代价，令其为 $f^*(s)$ 。我们有 $f(t) \leq f^*(s)$ 。
- (4) t 是最终目标结点， $h(t) = 0$ ，所以 $f(t) = g(t) + h(t) = g(t) \leq f^*(s)$ 。
- (5) $f(t) = g(t)$ 是一个可能解， $g(t) \geq f^*(s)$ ， $f(t) = g(t) = f^*(s)$ 。

A*算法本质的正确性

对于任意结点 n

- $g(n)$ = 从树根到 n 的代价
- $h^*(n)$ = 从 n 到目标结点的优化路径的代价
- $f^*(n) = g(n) + h^*(n)$ 是结点 n 的代价

定理1. 使用Best-first策略搜索树，如果A*选择的结点是目标结点，则该结点表示的解是优化解。

证明（反证法）：

- 假设A*算法首先找的路径 $P(n_1, n_2, \dots, T_p)$ 来到目标结点，而路径 P 并非真正最小代价路径。
- 若真正最小代价路径 $A(n_1, n_2, \dots, n_i, \dots, T_A)$ ，那么 A 中必有结点 n_i 不在路径 P 中，假设 n_i 是离起点最远的一个结点（路径 P 与路径 A 在 n_i 前面的结点都相同），那么 $f(n_i) \leq f^*(n_i) = g(T_A) < g(T_p) = f(T_p)$ ，即 $f(n_i) < f(T_p)$ 。
- 按照Best-First算法，如果 $f(n_i) < f(T_p)$ ，那么应该选择扩展 n_i 结点而非 T_p 结点，与Best-First策略相矛盾。故A*算法选择的结点是目标结点，则该结点表示的解是优化解。